

**AI-POWERED AUTOMATED DATABASE SCHEMA DESIGN
AGENT**

*MAJOR REPORT submitted in partial fulfillment of the requirements
for the Award of the Degree of*

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

Yasaswi Gorantla – 228W1A1221

Sai Charan Yaragorala – 228W1A1265

Under the Guidance of

Dr. Y. Sandeep

Assistant Professor



**DEPARTMENT OF INFORMATION TECHNOLOGY V R
SIDDHARTHA ENGINEERING COLLEGE
(AUTONOMOUS - AFFILIATED TO JNTU-K, KAKINADA)**

**Approved by AICTE & Accredited by NBA
KANURU, VIJAYAWADA-520007 ACADEMIC**

**YEAR
(2025-26)**

V.R. SIDDHARTHA ENGINEERING COLLEGE

(Affiliated to JNTUK: Kakinada, Approved by AICTE, Autonomous) (An ISO
certified and NBA accredited institution)

Kanuru, Vijayawada – 520007



CERTIFICATE

This is to certify that this internship report titled “**AI-POWERED AUTOMATED DATABASE SCHEMA DESIGN AGENT**” is a bonafide record of work carried out by **Yasaswi Gorantla (228W1A1221)** and **Sai Charan Yaragorala (228W1A1265)** under my guidance and supervision is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, **V.R. Siddhartha Engineering College** (Autonomous under JNTUK) during the year 2025-26.

Dr. Y. Sandeep

Assistant Professor

Dept. of Information Technology

Dr. M. Suneetha

Dean R&D, Professor &Head

Dept. of Information Technology

EXTERNAL EXAMINER SIGNATURE

Date of examination:

DEPARTMENT OF INFORMATION TECHNOLOGY

V.R. SIDDHARTHA ENGINEERING COLLEGE

PROJECT SUMMARY

S. NO	ITEM	DESCRIPTION
1	Project Title	AI-POWERED AUTOMATED DATABASE SCHEMA DESIGN AGENT
2	Student Names & Numbers	YASASWI GORANTLA (228W1A1221) SAI CHARAN YARAGORALA (228W1A1265)
3	Name of The Guide	Dr. Y. Sandeep
4	Research Group	Artificial Intelligence
5	Application Area	Intelligent Database Design Automation
6	Aim of the Project	To design an AI-driven system that converts natural-language software requirements into well-structured database schemas, identifies entities and relationships, generates SQL DDL scripts, renders ER diagrams, and validates the generated schema against expected outputs through dataset-based evaluation.
7	Project Outcomes	• Natural-language to schema conversion using an AI-assisted pipeline • Automated entity, attribute, and relationship extraction • SQL DDL generation and ER diagram creation • Explanation and suggestion modules for schema refinement • Dataset-based validation workflow for measurable evaluation

Student Signature

- 1.
- 2.

Signature of the Guide

ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **V.R SIDDHARTHA ENGINEERING COLLEGE** for giving me this opportunity for fulfilling my project. I am grateful to our principal **Dr. A.V. RATNA PRASAD**, for his encouragement and support all through the way of my project.

On the submission of this Project report, I would like to extend my honour to **Dr. M. Suneetha**, Dean R&D, Professor & Head of the Department, IT for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my project guide **Dr. Y. Sandeep**, Assistant Professor for his guidance and assistance in completing this project successfully.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	iii
LIST OF FIGURES	v
LIST OF TABLES	vi
ABSTRACT	viii
CHAPTER-1 Introduction	1
1.1 Origin of the Problem	1
1.2 Basic definitions and Background	2
1.3 Problem Statement with Objectives and Outcomes	5
1.4 Related Applications	9
CHAPTER-2 Review of Literature	12
2.1 Description of Existing Systems	12
2.2 Summary of Literature Study	16
CHAPTER-3 Proposed Method	20
3.1 Design Methodology	20
3.2 System Architecture Diagram	22
3.3 Description of Algorithms	25
3.4 Description of datasets, Requirements and Tools	35
CHAPTER-4 Results and Observations	41
4.1 Stepwise description of Results	41
4.2 Test case results/Result Analysis	54
4.3 Observations from the work	60
CHAPTER-5 Conclusion and Future work	64
5.1 Conclusion	64
5.2 Future study	65
References	68
APPENDIX	70

LIST OF FIGURES

Figure	Page No.
Figure 3.1: Architecture Diagram	24
Figure 4.1: Login Page Interface	40
Figure 4.2: IPLA Admin Interface	41
Figure 4.3: IPLA AI Chatbot Interface	42
Figure 4.4: Student Assistant Interface	43
Figure 4.5: URL-Based Content Processing	44
Figure 4.6: PDF Processor	45
Figure 4.7: Learning Assistant Dashboard	46
Figure 4.8: Courses and Materials View	47
Figure 4.9: Multilingual Interaction	48
Figure 4.10: RAG-Based Academic Response	49
Figure 4.11: Student Learning Dashboard	50
Figure 4.12: Confusion Matrix (Random Forest)	51
Figure 4.13: Model Performance Comparison	52
Figure 4.14: Schema Generation Interface	53
Figure 4.15: Extracted Entities and Attributes	54
Figure 4.16: Relationship Detection Output	55
Figure 4.17: SQL DDL Generation	56
Figure 4.18: ER Diagram Output	57
Figure 4.19: Validation Output	58

LIST OF TABLES

Table	Page No.
Table 3.1: Dataset Description and Domain Coverage	34
Table 3.2: Software and Tools Used in the System	36
Table 4.1: Model Performance Metrics Comparison (IPLA 2.0)	52
Table 4.2: Entity Extraction Validation Results	58

LIST OF EQUATIONS

Equation	Brief Description
Eq (1)	Requirement Input Representation
Eq (2)	Text Processing / Tokenization
Eq (3)	Entity Extraction Function
Eq (4)	Relationship Detection Function
Eq (5)	Schema Generation Mapping
Eq (6)	SQL Generation Function
Eq (7)	ER Diagram Graph Representation
Eq (8)	Similarity Calculation (Retrieval)
Eq (9)	Prediction Function (Random Forest)
Eq (10)	Accuracy Calculation (Validation)

ABSTRACT

The design of the database schema is a crucial phase of software development because it determines how. Application manages, stores and organizes data. Traditionally, schema design is done manually by studying requirement specifications, discovering entities and attributes, and creating relationships. This needs to be done in domain. is knowledge and can be time-consuming and error-prone. This project presents an AI-powered system for automatically generating database natural-language requirement-description schemas. Input is processed by the system. text, extracts entities and relationships and generates structured outputs, such as SQL. DDL statements and ER diagrams. It also gives validation through check against generated. entities with predictable outcomes in data-driven situations. The suggested method allows to perform a systematic change of informal requirements. to designed database design. The outputs created help users to comprehend. and polishing the schema prior to implementation. The results of the experiments indicate that the system can recognize core entities and. relations reasonably accurately. The system in the case under consideration has attained a accuracy of 75% in entity extraction. The results indicate that the system can serve as an effective aid in helping out with the initial phases of database design.

Keywords: database schema creation, ER, SQL DDL, entity extractor, natural language processing.

Chapter-1

Introduction

1.1 Origin of the Problem :-

In modern software development, the database plays a central role because it stores, organizes, and maintains the information required by an application. Whether the system is developed for a hospital, college, bank, or online business, the quality of the database structure directly affects the reliability and efficiency of the software. Before implementation begins, developers must decide what data needs to be stored, how different records are related, and what constraints should be maintained to preserve consistency.

Traditionally, this work is carried out manually by studying the requirement description and converting it into an Entity--Relationship model and relational schema. Although this approach is standard, it is often difficult for students and beginner developers. A textual problem statement may describe many operations, users, and transactions, but identifying the actual entities, attributes, and relationships from that description requires careful analysis and prior knowledge of database design principles.

In many academic and practical situations, developers understand the application idea but struggle to translate that idea into a proper database structure. They may miss important entities, choose unsuitable primary keys, or fail to represent relationships correctly. As a result, the generated schema may contain redundancy, weak normalization, or missing links between tables. These issues later affect query design, application logic, and overall system maintainability.

The problem becomes more challenging when the requirement description is lengthy, domain-specific, or written in an unstructured manner. In such cases, the database designer must interpret implicit information and infer hidden relationships that are not directly stated. This makes schema creation both time-consuming and error-prone, especially during the early stages of development.

Recent progress in artificial intelligence and natural language processing has created an opportunity to improve this process. Large language models and text analysis methods

have shown the ability to interpret unstructured input and produce meaningful structured outputs. This creates the possibility of building a system that can read a software requirement description, recognize important data objects, identify their relationships, and produce an initial database schema automatically.

The origin of the present project lies in this need for a practical tool that assists developers during the early design stage. Instead of replacing human judgment, such a system can reduce manual effort by generating a structured first draft of the schema. The user can then review, refine, and validate the output. In this way, the system acts as an intelligent design assistant that helps bridge the gap between natural-language requirements and formal database design.

1.2 Basic definitions and Background :-

It is required to have a clear idea of the basic concepts of databases before one can talk about the proposed system. As this project is concerned with the transformation of natural-language requirements into structured database artifacts, the fundamental notions of database design, schema generation, and language-based processing have to be interpreted in an appropriate way. Database schema design is commonly considered to be one of the initial phases of software development since it determines how application data is arranged, stored, and managed [1], [2].

A database schema is the logical organization or design of a database. It defines the tables, attributes, keys and constraints needed to store application data in a structured fashion [3]. An effective schema can be used to minimize redundancy, enhance consistency and facilitate efficient data retrieval. The schema provides the foundation of all the storage and retrieval operations in relational systems.

An entity is an object or concept in the real-life that information should be stored. Examples are student, doctor, patient, product and order. In conceptual modeling entities are the main building blocks on which database tables are based upon [4]. A set of attributes is present in each entity, an attribute being a property of that entity. To illustrate, a student entity might be in the form of student ID, name, branch and contact number.

Relationships explain the association of various entities with each other. An example is when a patient makes an appointment, a student enrolls in a course, and a customer orders something. These relations play a crucial role in schema design since they can be used in deciding how to connect the tables in the final relational model [5], [6]. When requirement descriptions are provided in natural language, it is a significant challenge to correctly determine such relationships.

Entity-Relationship model is an abstract method of representation of entities, attributes, and relationship prior to implementation [7]. It gives an overall perspective of the system and helps to move to formal requirements to formal database design. ER modeling is another of the most popular in the database design due to its ability to allow the user to see the organization of the data prior to creating SQL code.

The primary key refers to an attribute, a group of attributes, which make each record in a table unique. A foreign key is a characteristic of one table that points to the primary key of another table hence developing a relationship between two tables. These keys are essential in ensuring uniqueness and referential integrity in relational databases [8].

Normalization refers to a process of arranging data in a relational database where the redundancy is minimized and the data is not subject to anomalies when inserting data, deleting it, and updating [9]. It is a general rule of thumb of relational database design and enhances maintainability and data consistency. Normalization is useful in real world systems to make sure that the database structure does not change as the application increases.

SQL Data Definition Language SQL Data definition language is a language like CREATE TABLE, ALTER TABLE and DROP TABLE which are used to create and alter database structure. After the schema has been ready conceptually, it is implemented in a relational database system using the following commands. Hence, schema generation can only make sense when it can be realized into valid SQL definitions.

Ideas of natural language processing and large language models are also relied upon in the current work. Textual input is usually analyzed by natural language processing

techniques to reveal meaningful patterns, and to extract structured information from unstructured description [10]. As part of this project, this processing assists in extracting candidate entities, descriptive attributes, and relationship indicators out of requirement text.

Recently, large language models have demonstrated high performance in contextual understanding, the ability to produce structured output, and software engineering aid [11], [12], [13], [14], [15]. Compared to purely rule-based systems, these models are able to draw meaning out of requirement descriptions more effectively, and thus are appropriate when it comes to tasks like entity extraction, relationship identification and schema suggestion. They are used in this project because they are necessary to fill the gap between the informal requirement statements and formal relational design.

Entity extraction is the process of recognizing crucial domain objects in natural-language input [16]. Relationship detection is the determination of the interactions of the extracted entities with one another in the described system [17], [18]. Combined, the two operations constitute the essence of automated schema generation. After identifying entities and relationships, the system may proceed to schema synthesis, SQL generation and creation of ER diagram.

The checking of the output generated against expected design components is known as schema validation. Validation is conducted in this project by comparing the generated entities with expected entities based on curated examples. This provides a quantifiable means of estimating the effectiveness of a system and assists in determining whether the generated schema is an accurate representation of the original requirement [19], [20].

The history of this project is at the crossroads of database design, natural language understanding and AI-assisted software engineering. Conventional approaches consider requirement analysis and schema creation as manual and independent activities. Nevertheless, the recent advances in NLP and language models indicate that the requirement text itself can be regarded as a structured source of database design support [1], [3], [7], [11]. This is the understanding which is the foundation of the proposed system.

1.3 Problem Statement with Objectives and Outcomes :-

1.3.1 Problem Statement :-

One of the most significant phases in software development is database design since it defines how the data in applications will be represented, related and maintained over the life cycle of the system [1], [2]. Although the functional concept of an application is evident, a written requirement description can hardly be translated into a correct relational schema. It involves the designer to find some basic entities, define appropriate attributes, create keys, and define appropriate relationships between tables. In development environments of both academic and beginner levels, this process may be done manually, and therefore is slow, inconsistent and highly individual dependent.

One of the challenges in manual schema design is that natural-language requirements tend to be informal, and not structured. This requirement might be taken differently by various developers resulting in various database design. Other designers might fail to consider the important entities and others might add unnecessary tables or even fail to get appropriate cardinalities. These anomalies compromise the design quality and in most cases lead to issues in the subsequent implementation phases [3], [4], [5].

The other issue is that most of the existing systems and tools do not directly address the issue of schema generation given requirement text. There are those tools which facilitate the manual creation of ER diagrams and there are those which create SQL once the schema has been predefined by the user. Text-to-SQL systems have demonstrated good improvement in translating natural-language queries into SQL statements, but typically it is assumed that the database schema is already known [6], [7], [8]. This puts them inappropriate to early-stage software design, where the schema itself has not yet been developed.

The recent studies have demonstrated that big language models and systems based on NLP can help extract useful structured facts out of the textual input [9], [10], [11], [12], [13], [14]. Nevertheless, most of these methods continue to have significant drawbacks. There are those that are highly reliant on immediate words, those that

need more than one agent or intricate correction pipelines and those that do not give complete outputs like SQL schema, ER visualization, and validation within a workflow [15], [16], [17], [18]. Consequently, a gap exists between scholarship-based schema generation approaches and a feasible, student-friendly system which could be utilized in academic project work.

The other critical concern is that automated tools lack the support of explanation and review. Most times a system will produce an output without the user being made aware of the manner in which that output was obtained. This is an issue particularly to students, since it is not only important to be provided with the result but also the logic behind the generated schema. An effective system must not only successfully automate schema generation, but also ensure that the output is comprehensible, readable, and refinable.

The lack of an easy, combined, and pragmatic mechanism that is capable of processing a natural-language software requirement description as input and producing meaningful database design artifacts is the problem to be solved in this project. These artifacts are expected to contain extracted entities, identified relationships, SQL DDL statements, ER-style visual output and quantifiable validation feedback. The benefits of such a system must include reducing manual work, enhancing uniformity and offering a well-organized starting point to schema design without fully eliminating the human review.

Here, the current project attempts to solve the challenge of converting informal textual specifications into formal relational design in a manner that is comprehensible, modular, and applicable in either academic or prototype level applications [19], [20], [1], [2]. The suggested solution is not merely about automation, but interpretability and validation as well, to ensure that the resulting schema is a sound first draft and not an opaque end product.

1.3.2 Objectives :-

The project objectives are the targeted goals that will guide the design, development and implementation of the proposed system both technically and practically. The overall aim is to create an AI-based automatic database schema design system, which

can translate natural-language software requirement descriptions to a structured relational database artifact. The system is supposed to make the work of designing schema by hand a little bit harder and offer a more effective method to transfer the informal descriptions of problems directly to the formal database schemas.

The other significant goal is to determine and retrieve core entities, attributes and relationships in unstructured textual input. As the software requirements are often formulated in plain language, the project will attempt to process this input in an intelligent way and convert it into useful schema elements. This goal is significant since proper extraction of entities and relationships is the basis of proper database design.

One of the goals of the project is to produce SQL Data Definition Language statements of the extracted schema. The system must not merely conclude with the conceptual analysis but must also deliver implementation oriented outputs like relational tables, primary keys and foreign key constraints. This helps the proposed system to facilitate not just academic knowledge but also practical software development.

The other significant goal is to generate ER-like visualizations of the generated database schema. The system will simplify schema validation, refinement, and understanding by providing entities and their relationships in a graphical format. This goal enhances the ease of use of the system and assists users to check the accuracy of the produced design.

Another goal is that it must incorporate validation support through comparison of generated entities to expected entities of curated datasets. This enables the system performance to be measured by means of measurable criteria and also in determining the degree at which the produced schema is a good representation of the requirement description.

The other realistic purpose is to have a convenient interface where user input the software requirement descriptions and receive systematic outputs in form of entities, relationships, SQL schema and diagrams. This goal will make the system accessible, interactive and helpful to students, researchers as well as novice developers. Another purpose of the project is to develop an entire end-to-end workflow, where the requirement analysis, entity extraction, relationship detection, schema generation,

visualization, and evaluation can be put into one framework. This objective makes the system more comprehensive than conventional approaches that deal with these tasks separately.

Finally, the project as a whole consists of the development of a useful intelligent design assistant that eases the process of designing a database schema at an early stage, enhances design consistency and aids both learning and prototype level development through automation, visualization and evaluation.

1.3.3 Outcomes of the Project :-

The outcomes of the project serve as a measuring rod of the effective accomplishment of the proposed goals both technically and practically. The main deliverable is the creation of an automated database schema design system powered by AI and capable of converting natural-language software requirements descriptions into structured relational design artifacts. The system will be able to extract core entities, attributes and relationships based on textual input hence, it will minimize the manual effort of the initial stages of database design.

A second significant consequence is that the meaningful schema components can be extracted effectively out of unstructured requirement text. The system will then be able to identify the candidate entities, supporting attributes and relationship patterns by examining the linguistic and contextual data in the input, and these are critical in constructing a relational database structure. This enhances schema preparation consistency and speed as opposed to a completely manual method.

One of the notable findings of the project is that the schema extracted was successfully used to generate SQL Data Definition Language statements. The system will translate conceptual design items into implementation-related deliverables like table definitions, primary key, and foreign key constraints. This renders the schema created more viable in the real-world software development and prototyping.

The other significant consequence is that ER-like visual representations of the schema generated are created. Such diagrammatic outputs will help simplify the database structure and make it easier to validate and refine it. The system enhances

interpretability by visually defining objects and their connections, and helps with academic study and design inspection.

Another result is the incorporation of validation aid by comparison of generated entities and anticipated entities based on curated datasets. This makes the performance of the system to be quantified in a structured criteria like extraction accuracy. Consequently, the project is likely not only to offer automated output generation, but also a quantifiable means of evaluating the quality of the generated schema.

The other practical result is that a user-friendly interface has been developed with the help of which users can input descriptions of requirements and receive generated outputs in the form of extracted entities, relationships, SQL schema, and visual diagrams. This enhances usability and makes it easier to access the system to the students, novice developers and even prototype level software developments.

It is also anticipated that the project will result in the creation of an entire end-to-end workflow that incorporates a requirement analysis, entity extraction, relationship detection, schema generation, visualization, and evaluation within a single system. This renders the suggested approach more holistic than conventional methods that consider discrete tasks like ER drawing or SQL generation.

Finally, the last deliverable of the project is a detailed and feasible intelligent system that can translate natural-language specifications to sensible database design products. It was anticipated that the work would help in the automation of database design, enhance the efficiency of early-stage software development, as well as offer an efficient academic support tool to learn how to create relational schemas.

1.4 Related Applications :-

AI-based automated database schema design agent can be applied in a variety of ways, both academic and practical. Because database design is an aspect of a system that is often needed even in many types of software systems, a tool that is capable of converting natural-language requirements into structured schema output can be useful in a broad variety of fields. The applicability of the suggested system is not restricted

to a particular use case as nearly all information systems rely on a structured database scheme to store and handle data.

One of the most important applications of the proposed system is in academic project development. Students in engineering and computer science education are frequently presented with problem statements in natural language and then asked to derive ER diagrams, relational schemas and SQL definitions. This step is challenging to many students as it involves both conceptual knowledge and actually designing. The system may in this case be used as an academic support tool, by providing a first draft of the schema based on the project description. This will assist the students in seeing the relationship between requirements and database design and will save time spent on manual trial and error preparation.

The other significant use of the system is in rapid prototyping to develop software. At the initial phases of a project developers and teams can require a fast and reviewable database structure to allow implementation to commence immediately. The proposed system may be useful in such cases to generate an initial schema, SQL DDL statement and relationship mappings based on a short textual requirement description. This saves time in between requirement analysis and database set up to enhance development efficiency.

The system can also be employed in a multi domain database design. As it supports natural-language input, it can be used on various types of applications e.g. a hospital management system, e-commerce platform, school administration system, inventory management application, hotel booking system, banking solution and library system. This flexibility enhances the flexibility and usefulness of the system, since it is not bound to a certain domain or preset schema structure.

The other related usage is in requirement analysis and design support. In most software projects the most difficult part is not the implementation but to get the data requirements right. The suggested system is able to guide the analysts and developers by revealing the probable entities, attributes and relationships concealed within the requirement text. This is in turn useful as a design-support tool at the planning stage of software

development where the teams require to confirm that the description of the problem has been understood properly.

Applications in teaching and learning database concepts can also be found in the system. It can be used as a demonstration tool by faculty members when teaching topics like ER modeling, entity extraction, relational schema design, and SQL generation. The system will be able to enhance conceptual understanding among students by demonstrating how a plain-language problem statement can be converted into structured database outputs. In such a manner that it is not only a technical tool but also an educational aid.

The other significant use is in research and benchmarking. The system has a validation element which matches the generated entities with those in the curated datasets. This renders it applicable in the research studies associated with automated schema generation, requirement engineering, and AI-assisted software design. Such a system can help researchers to test various approaches, compare the accuracy of extraction, and discuss the advantages and weaknesses of the natural-language-based schema generation techniques. The system also can be used as a guide to novice developers and small software teams in the real world. Small projects do not necessarily have trained experts on databases and the developers might require assistance on the design phase. The proposed system has the ability to give us an organized beginning by creating a draft of a first level schema, which can subsequently be refined according to project requirements. This assists in minimizing mistakes and enhancing uniformity in preliminary design choices.

On balance, the suggested system is applicable in any place where it is necessary to convert textual requirements in software into useful products of database design. It can be used in academic learning, prototype creation, multi-domain software design, requirement analysis, teaching support, and experimental evaluation. Due to such a broad applicability, the system may be regarded as a practical development tool as well as an effective educational and research one.

CHAPTER-2

Review of Literature

2.1 Description of Existing Systems :-

A number of currently used systems and research methods have tried to solve portions of the problem of converting natural-language input into structured database-related output. Most of these systems however do not offer an end-to-end solution but instead offer a particular part of the system like entity extraction, SQL generation, requirement analysis or schema correction. The significance of the investigations of these systems is that it can assist in realizing the present state of the research and the constraints that drive the proposed work.

Text2Schema is one of the key current methods that generate relational schema formats based on descriptions in natural language through the use of large language models and post-processing techniques that correct the generated output [1]. This system goes to show that language models can be used to recognize database components out of textual specifications. Nevertheless, it greatly relies on quality and clarity of the description of input. In case the requirement statement is not clear and/or complete, the generated schema can also be inconsistent.

Another similar system is that of SchemaGenerator known as a multi agent design approach to schema generation tasks [2]. The planning, schema reasoning and evaluation are conducted by different agents in this approach. This type of decomposition enhances the quality of reasoning and streamlines the process of generation. Such a system, however, adds to the further architectural complexity and computation cost. In lightweight academic use, this is not necessarily practical.

Another body of work synthesizes natural language processing methods with language models at scale to extract entities and produce schema-oriented results [3]. These systems apply text analysis techniques to identify candidate entities, attributes and relationships and then transform them into a structured schema. Though this enhances interpretability, these systems are still reliant on high-quality input phrasing and might

need human intervention in situations where the requirement has implicit domain information.

The literature contains many papers dedicated to text-to-SQL systems, in which the goal is to transform natural-language queries into executable SQL statements [4]. Such systems have demonstrated great improvement using transformer-based models and benchmark datasets. However, they generally assume that the underlying database schema already exists. Due to this, they do not lend themselves to requirement-driven schema generation, where the schema itself needs to be generated prior to query execution making any sense.

Another existing approach is Gen-SQL, which uses iterative refinement to improve the quality of generated SQL outputs [5]. The system engenders a generate-ground-regenerate cycle so as to minimize structural errors and enhance correctness. Although this enhances the quality of SQL, emphasis is laid on generating a query instead of the entire schema. Thus, it can be considered helpful in structured output generation, but has limited value when it comes to creating database schemas.

Systems founded on more traditional database design techniques and conceptual modelling systems are also to be found, in which the developer transforms the requirements to ER diagrams and relational schemas manually [6]. These are the methods that are academically powerful and well developed but are totally reliant on human effort. They also lack automated support in the event of large or unstructured requirement descriptions.

A number of publications in the field of requirement engineering apply NLP methods like tokenization, part-of-speech tagging, noun phrase extraction, and dependency parsing to process software requirement documents [7]. These techniques can be employed in the recognition of candidate objects and actions based on textual descriptions. Nevertheless, rule-based or parsing-based systems can easily falter when the input is ambiguous, has a complex sentence form, or has implied relationships.

Other systems add knowledge-assisted or ontology-based methods to enhance the quality of schema generation [8]. These systems can understand more context-specific words with the addition of domain knowledge and minimize extraction errors.

Nevertheless, these systems demand managed ontologies or bespoke knowledge bases, and thus restrict their adaptability to various areas. Such domain knowledge might not necessarily be provided in practice student projects.

The study of explainable AI systems in the software engineering field has also affected the design of recent intelligent tools [9]. The goal of these systems is to provide reasoning traces or explanations and generated outputs in order to allow the users to know how conclusions are made. Although explainability enhances trust and reviewability, most of the current solutions focus on explainability rather than fully generating artifacts. Consequently, they might not completely endorse automated schema design.

Another type of existing tools is graph-based modeling and visualization systems [10]. These systems are very useful to depict entities and relationships graphically in ER diagrams or conceptual graph. They however normally ask users to define the schema manually prior to visualization. Thus, they can be used as a representation, however, not to convert text into schema structures automatically.

Recent research on large language models of code and software tasks indicates that large language models may be useful in software engineering tasks such as design reasoning, and artifact generation [11]. Such systems exhibit good contextual knowledge, and it applies to schema design tasks. Nevertheless, raw model outputs might still need further validation and domain-specific refinement.

Other current literature in semantic evaluation and generation that is motivated by benchmarks has suggested approaches to the quantification of correctness of structured output against predicted output [12]. These strategies are valuable as they emphasize that evaluation must be quantifiable rather than based solely on visual examination. Such evaluation methods can be used in the context of schema design in validation of whether the generated entities and structures are a match of the expected schema components.

The other significant group is few-shot and pre-trained language models that are capable of producing technical content based on brief prompts [13]. The systems have greatly enhanced the capacity of machines to read between the lines and produce

structured textual materials. This impact can be seen in a variety of contemporary design-assistance systems. Nevertheless, they are powerful in general reasoning, they might nevertheless have difficulty with the domain-specific schema constraints and implicit relationships when applied without an extra structure.

Beyond these, more recent methods have considered retrieval-augmented generation, knowledge-grounded structured output system, interactive design assistants to enhance reliability in software engineering tasks [14]. These systems take a combination of retrieved context and generative reasoning and produce more grounded outputs that are less reliant on raw prompt-based generation. These approaches are applicable since they indicate how schema design systems can be made more reliable with the assistance of external knowledge or instances.

The research on automated conceptual modeling of requirement text also exists, where the systems are trying to map requirement statements to conceptual objects, associations, and design elements [15]. These approaches are directly connected to the suggested work as they solve the issue of the disconnect between text and schema. Nevertheless, most of them are still incomplete solutions and lack executable SQL, diagram generation and validation within a single workflow.

Other researchers have explored hybrid AI pipelines consisting of a combination of NLP, transformer models, and rule-guided structural reasoning to generate design artifacts more effectively [16]. Such systems have better consistency than all-generative approaches, although they usually are experimental and not tuned to accessible pedagogical applications.

Lastly, some current systems focus on human-in-the-loop design assistance in which automatically generated outputs are not intended to be final schema solutions but to be reviewed as drafts [17]. This concept comes in particularly handy in academic and prototype-level contexts, where the automation does not aim at eliminating human judgment, but at decreasing the amount of manual effort and enhancing the quality of the initial design.

In general, the overview of the current systems reveals that significant advances have been achieved in the fields of natural-language interpretation, entity extraction, SQL

generation, explainability, and visualization. Nevertheless, the majority of the systems available tackle only single aspects of the issue. Few systems will offer a single workflow that takes in requirement text, extracts schema elements, creates SQL, produces ER-like visual output and validates the output in a single framework. It is this gap that will present the incentive of the suggested AI-based automated database schema design agent.

A cursory examination of previous work reveals that database design support has progressively shifted away from completely manual modeling to the AI-aided generation techniques. With older methods, developers were required to read requirement documents, determine the entities, make a decision about the attributes, and create the ER diagrams and relational schema manually. Since such a process largely relies on the subjective comprehension, numerous researches have noted that manual design is time-consuming and highly inconsistent, in the case of large or ambiguous requirements. In more recent work, emphasis has moved onto systems based on natural language processing and large language models to minimize this effort. The more recent approaches aid software interpretation of requirement text, interpreting written descriptions and relating written descriptions to structured schema deliverables [18], [19], [20].

2.2 Summary of Literature Study :-

A cursory glance at previous work reveals how the concept of database design support has progressively transitioned away from exploring entirely manual modeling, to AI-assisted generation approaches. In the ancient methods, developers were forced to read requirement documentation, determine entities, make decisions on attributes, and create hand-drawn ER diagrams and relational schemas. Due to the fact that this is a process that is largely reliant on individual knowledge, most studies identified that manual design is time-consuming and rather inconsistent in cases where requirements are few or vague. Even more recent studies have been moving to systems that rely on natural language processing and large language models to minimize this effort. The more recent

approaches assist software to make more sense out of requirement text and relate written descriptions to formal schema results [1], [2], [3].

A major trend in the literature is the increasing popularity of language-based analysis as a way of deriving database entities and relationships based on requirements descriptions. Rather than employ pure keyword detection, more recent methods attempt to gain insight into the meaning of nouns, actions, and dependencies found in the text. Due to this change, systems are becoming more efficient in identifying candidate entities, associating attributes, and mapping relationships in a more meaningful manner. According to NLP pipelines and LLM-assisted extraction, this type of technique can enhance the quality of schema generation particularly in comparison to more basic rule-based techniques [4], [5]. Simultaneously, the literature also helps to understand that these systems remain dependent on the quality of the input and can be put to the test in the situations when the information is implied instead of being explicitly given.

The other notable finding of the current literature is the level of impact of text-to-SQL studies to this field. Natural-language instructions can be translated into SQL queries with high precision using many modern systems and this demonstrates that a structured output can be generated using text, which is technically feasible [6], [7], [8]. The interesting fact is, though, that the majority of these systems presuppose the existence of the database schema. Due to this, they can be used in query generation but not completely applicable to early-stage schema design, where the schema itself needs to be generated. This is why schema generation is a more challenging issue than query generation as it involves determining the structure before a query can be written.

In further examining the newer systems, it can be seen that there is an increasing trend of multi-stage and hybrid approaches. There are studies involving correction pipelines, and there are studies that split the work into planning, generation and validation stages [9], [10], [11]. This indicates that people have come to the realization that schema generation is not always able to be well managed in a single step. Combining several steps tends to enhance consistency, minimize errors, and generate easier-to-validated

outputs. Nonetheless, a great number of them are still complicated, resource-intensive, or hard to implement in student-level and prototype-based environments.

The other obvious trend in the literature is the significance of conceptual database theory in the development of automated techniques. Although the more recent systems rely on AI, they continue to rely on historical concepts like entities, attributes, keys, normalization and relationship mapping [12], [13]. This implies that smart schema generation does not overrule the design concepts of databases, but instead extends them. That is, the more powerful systems tend to be those that integrate text understandings with sound relational design principles as opposed to free-form generation.

Other research topics that have emerged in the literature include the growing interest in explainability and review support in AI-assisted software systems. According to several studies, the outputs generated are more helpful when users are able to comprehend how the system got them [14]. This is of particular importance in schema design where students and developers must ensure that the extracted entities and relationships accurately represent the original need. A system that simply generates output but does not tell us why, can save time, yet this is not always useful in getting the users to learn or trust the outcome. This has led to the fact that explanation oriented support is now regarded as a desirable attribute in intelligent design tools.

The retrieval-based and knowledge-grounded generation methods are another beneficial direction observed in the past work. Integrated generation and retrieved evidence or structured support systems tend to generate more stable and context-aware outputs [15], [16]. This is important since a general weakness of purely generative models is that they could create partial or unsustained schema elements in the face of a vague requirement. The idea of basing generation on external support is useful in mitigating this issue and is oriented towards more robust design support.

Despite these developments, most of the existing systems still address only a single aspect of the issue. Others are good at entity extraction, others at SQL generation, others at conceptual modelling, and others at explanation or visualization. Few include all these functions in a single workflow. This has resulted in an empirical disconnect between research advancement and a system capable of taking requirement text,

recognising schema elements, creating SQL, generating ER-style results and aiding validation in an integrated manner [17], [18], [19], [20].

Altogether, the literature shows only one thing: semantic interpretation, structured generation, conceptual modeling, and explainable AI support have been developed intensively, but little has been done in integrating these aspects. Previous research is a solid argument that it is feasible and practical to generate database schema automatically. Meanwhile, they too demonstrate that a single intelligent framework, which relates requirement analysis, entity extraction, relationship detection, SQL generation, visualization and validation in a single practical system, is still needed. The results are directly related to the creation of the suggested AI-based automated database schema generator.

CHAPTER-3

Proposed Methodology

3.1 Design Methodology:

The design approach of the offered system is the structured and sequential workflow transforming natural-language software specifications into formal database design artifacts. As opposed to creating the entire schema at once, the system operates in phases so that a given phase is tasked with a certain task and the output of each stage is fed into the next stage. This approach enhances understanding, minimizes design uncertainty and simplifies the whole process to comprehend, validate and revise.

The methodology starts with the user collecting the requirement description. The input can be in the form of a plain-language software description or as a dataset entry to be evaluated. The first goal of the methodology is to transform this unstructured text into a processable one since requirement statements are typically written informally and can include a combination of domain objects, actions and conditions. This is significant since the quality of the end schema is highly dependent on the level of accuracy of the requirement interpretation in the initial phase.

Once the input is received, the system processes it with a text analysis to establish meaningful linguistic units like nouns, noun phrases, verbs, ownership words, and dependency phrases. Nouns tend to nominate potential entities and descriptive phrases tend to indicate characteristics. Likewise, the verbs and expressions that are related to the action are also useful in identifying the relationship among entities. This step will form the conceptual basis of the subsequent schema design process since it will convert the requirement statement to a candidate design element set.

The second step of the methodology is concerned with the entity and attribute formation of candidates. At this phase, the system identifies potential domain entities in the requirement text and collects similar descriptive information below them as attributes. The first pass is a deliberate extraction step that is broad to ensure that important

elements are not overlooked too soon. This enables the system to develop a more detailed representation of the initial design prior to the refinement process.

After identifying entities and attributes, relationship discovery and structural interpretation become the next steps of the methodology. At this point the system learns about the relationship of the extracted entities with each other. It analyses the textual arrangements that indicate the associations of booking, managing, containing, storing, recording, or assigning. Depending on these interactions, the system determines the kind of relationships that include one-to-one, one-to-many or many-to-many. This step is extremely critical as at this point the database design involves not only the identification of data objects but also the interaction between these data objects within the system.

Once the conceptual structure is ready, the methodology goes on to schema synthesis. Here, the entities extracted are converted to relational tables, attributes converted to columns and appropriate keys are given. Primary keys are used to define uniqueness, and foreign keys are added in those places where entities relationships are required to be upheld. In case of complex associations like many-to-many mappings, the system forms linking structures in between. This is the phase that transforms the idea into an adequate relational design.

SQL generation is the next step in the methodology. When the schema structure is complete, the system generates SQL Data Definition Language statements that characterize tables, attributes, primary keys and foreign key constraints. This is a realistic phase in the working process since it transforms the created design into a form that can be immediately utilized in implementing in a relational database system.

The approach also features ER-style diagram rendering in addition to SQL generation. This step is aimed at giving a graphical representation of the database schema to allow users to easily understand and check the extracted entities and relationships. The visual representation enhances better interpretation and enables one to detect missing connections, faulty mappings, or structural discrepancies.

Validation and review is the last phase of the design methodology. The expected outputs are present to the system, which compares the generated entities with

reference entities and calculates evaluation outputs like the accuracy of extraction. This enables the effectiveness of the system to be quantified in an organized manner. The review stage also assists users to review the generated schema, what it is composed of, and refines it further should the need arises.

As a whole, the system design methodology of the proposed system is systematic and is conducted in a pipeline where the requirement acquisition process is intertwined with the process of text analysis, entity extraction, relationship detection, schema generation, SQL creation, visualization, and validation. The approach renders the system more dependable, more decipherable, and more appropriate to both academic and prototype-level database design applications.

3.2 System Architecture Diagram:

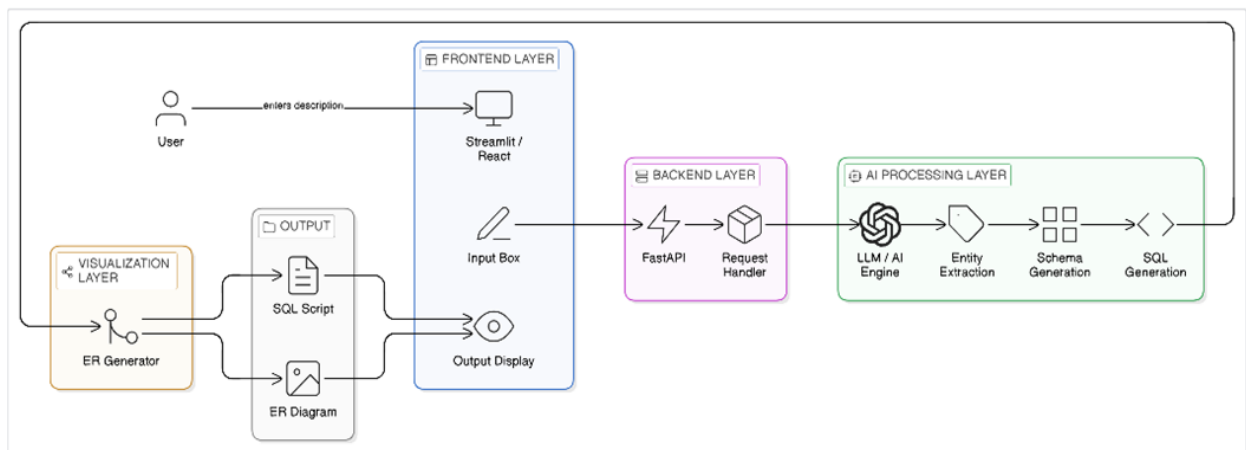


Figure 3.1: Architecture Diagram

Figure 3.1 provides the overall structure of the suggested system that is a layered and multi-stage pipeline that aims at transforming natural-language software requirement descriptions into structured database design outputs. The architecture is a combination of various noteworthy functions, including user interaction, requirement analysis, entity and relationship extraction, schema synthesis, SQL generation, ER-diagram visualization, and validation. The system is planned in a linear but modular fashion with each phase contributing to the overall workflow but still permitting the future improvements to be made without touching the whole design.

The architecture starts with the user input layer whereby the system takes software requirement specifications in natural language. These descriptions can be typed in by the user or can be chosen in a dataset that is employed in evaluation. This step is significant since it is where informal textual information is absorbed into the system. The description of the requirement is very critical in the quality and clarity of the latter stages as the whole further processing process is based on the effectiveness of the initial statement of the problem to reflect the desired system performance.

Once the input is received, the architecture enters into the stage of requirement processing and analysis. Here, this section of the system analyzes the text to locate significant linguistic elements like nouns, noun phrases, verbs and context expressions. Nouns tend to point out what could be, whereas descriptive words are attributes. On the same note, dependency patterns and action-oriented words are useful in determining interactions among entities. This step is vital since it transforms unstructured requirements text into a form that can be employed to construct a schema.

The second key element in the architecture is entity and attribute extraction module. At this level, the system recognizes potential domain objects that exist in the requirement description and clusters the corresponding description information. These are the candidate entities that constitute the conceptual database schema. The extraction process is extensive enough to include both a clearly stated and contextually important elements and thus less even minor components may be overlooked in the initial phase of design.

After extraction of entities, the architecture is passed through the stage of relationship detection. During this phase, the system looks at the relationship between the extracted entities with each other according to the actions, dependencies and operation patterns that have been described in the requirement text. Types of relationships which include one-to-one, one-to-many and many to many are deduced based on the language used in the input. The stage is very crucial since a database schema can only be meaningful when the connections between its entities are appropriately defined and represented structurally.

Once the conceptual structure has been created, it passes on to the schema generation module. In this stage, the entities and relationships identified are transformed into relational database formats. Tables are used to map entities, columns to map attributes, and keys are used to ensure uniqueness and referential integrity. When there are many-to-many relationships, linking tables are created by the system in order to keep the relational structure logically valid. This step converts the conceptual interpretation to a formal schema representation.

The next important part of the architecture is the SQL generation module. This phase involves translating the relational schema developed in the last module into statements of the SQL Data Definition Language. These statements contain commands to create a table, define an attribute, define a primary key and define a foreign key constraint. This step is especially helpful since it transforms the resulting schema into the form that can be easily read or entered into a database management system.

The architecture also has a visualization module which produces ER-style diagrams along with SQL generation. This element shows the extracted entities and relationships in a graphical representation that simplifies the structure of the structure to be easy to understand and validate. The visual output will enable users to ensure that the system has understood the description of the requirement and that the schema generated reflects the desired data structure. The stage will enhance interpretability and make the system more practical in both academically and prototype-level situations.

Another important component of the architecture is the validation and evaluation module. In the case of the use of dataset-based examples, the generated entities will be compared to the expected entities, which are present in the reference data. According to this comparison, the system computes the evaluation results like the extraction accuracy. This phase enables measurement of the performance of the system in an organized manner and assists in how well the system can transform text of requirements into meaningful schema outputs.

Lastly, the modules are all combined into a single end-to-end workflow with the input requirement description being transferred to analysis, extraction, schema generation, visualization, and validation in a linked order. Meanwhile, the architecture is modular

in nature, so that the enhancement or replacement of separate components can be effectively done. This gives the system the ability to be flexible, scalable and able to be improved in the future to include more functionality like improved relationship detection, normalization support, multilingual requirement support and more robust validation procedures.

Altogether, the offered system architecture offers a full and systematized structure on how to convert natural-language requirements into structured database schema artifacts. The architecture integrates language analysis, relational modeling, SQL generation, visualization, and evaluation into a single workflow to overcome the shortcomings of separate current methods and provide a useful solution to AI-assisted database schema design.

The presented work employs a series of algorithms to process user input, derive meaningful information, produce artifacts of the database schema and validate the results produced. Because the entire project encompasses both previous phases of intelligent learning assistant and the eventual schema design agent, the algorithm section is the discussion of the key computational modules which are deployed within the system. These algorithms are retrieval-based academic response generation, PDF and URL content processing, predictive analytics, multilingual query processing, entity extraction, relationship detection, schema generation, ER diagram generation and dataset-based validation. These algorithms make up the entire logical workflow of the system.

3.3 Description of Algorithms

3.3.1 Retrieval-Augmented Academic Response Algorithm

The retrieval-augmented academic response algorithm is used in the Intelligent Personal Learning Assistant phase of the project. Its purpose is to generate context-aware answers by combining user queries with relevant information retrieved from processed documents. Instead of producing responses only from the language model,

the algorithm first searches for the most relevant document segments and then uses those segments as context for answer generation.

Algorithm:

Input:

The input consists of a user query q and a collection of processed academic documents. These documents are divided into smaller chunks for effective storage and retrieval. The document set is represented as:

$$D = \{d_1, d_2, \dots, d_n\} \quad (3.1)$$

Equation (3.1), $D = \{d_1, d_2, \dots, d_n\}$, represents the set of document chunks derived from the processed academic documents. Here, D denotes the complete document collection and each d_i is an individual chunk. The purpose of this equation is to organize large documents into smaller units, enabling efficient storage and fine-grained retrieval.

Each document chunk is converted into an embedding vector:

$$\vec{E}_i = \text{Embed}(d_i) \quad (3.2)$$

Equation (3.2), $\vec{E}_i = \text{Embed}(d_i)$, defines the embedding of each document chunk. Here, d_i is a document chunk and $\vec{E}_i$ is its vector representation. The function $\text{Embed}()$ converts text into a numerical vector capturing semantic meaning. The purpose of this equation is to represent document content in a form suitable for similarity comparison.

Method:

When a user submits a query q , it is also converted into vector form:

$$\vec{E}_q = \text{Embed}(q) \quad (3.3)$$

Equation (3.3), $\vec{E}_q = \text{Embed}(q)$, represents the embedding of the user query. Here, q is the user query and $\vec{E}_q$ is its vector form. The purpose of this equation is to transform the query into the same vector space as document embeddings, enabling semantic matching.

The system computes similarity between the query embedding and the stored document embeddings to retrieve the most relevant content. Let the retrieved document set be

represented by R . The final response is then generated using a language model conditioned on both the query and the retrieved context:

$$A = \text{LLM}(q, R) \quad (3.4)$$

Equation (3.4), $A = \text{LLM}(q, R)$, defines the response generation step. Here, q is the user query, R is the set of retrieved relevant document chunks, and A is the generated response. The purpose of this equation is to produce a context-aware answer by combining the query with retrieved knowledge using a language model.

Output:

The output is a contextually grounded academic response A , generated from both the user query and the most relevant retrieved document segments.

3.3.2 PDF Processing Algorithm

The PDF processing algorithm is used to convert uploaded PDF documents into searchable and queryable knowledge sources. It enables the system to extract textual information from academic documents and prepare them for retrieval-based interaction.

Algorithm:

Input:

The input consists of uploaded PDF documents containing textual information.

Method:

The document is first parsed to extract text. The extracted text is divided into smaller chunks for efficient processing:

$$C = \{c_1, c_2, \dots, c_n\} \quad (3.5)$$

Equation (3.5), $C = \{c_1, c_2, \dots, c_n\}$, represents the set of text chunks obtained after extracting content from the PDF. Here, C denotes the complete chunked text, and each c_i is an individual segment of the document. The purpose of this equation is to divide large unstructured text into smaller, manageable units, enabling efficient processing and improving retrieval accuracy.

Each chunk is embedded into vector form:

$$V_i = \text{Embed}(c_i) \quad (3.6)$$

Equation (3.6), $V_i = \text{Embed}(c_i)$, defines the embedding of each text chunk into a vector form. Here, c_i is a text chunk and V_i is its corresponding vector representation. The function $\text{Embed}()$ converts textual data into a numerical format that captures semantic meaning. The purpose of this equation is to transform text into embeddings that allow similarity-based search and efficient storage in the retrieval system.

These vectors are stored in a retrieval structure so that relevant chunks can later be identified in response to user queries.

Output:

The output is a structured and searchable representation of the uploaded PDF document.

3.3.3 URL Content Processing Algorithm

The URL content processing algorithm enables the system to fetch, clean, and process web-based textual content so that it can be used for question answering and academic assistance.

Algorithm:

Input:

The input consists of a URL entered by the user.

Method:

The system fetches the webpage content and removes irrelevant HTML elements. The cleaned text is segmented into smaller units:

$$W = \{w_1, w_2, \dots, w_n\} \quad (3.7)$$

Equation (3.7), $W = \{w_1, w_2, \dots, w_n\}$, represents the set of text segments obtained from the cleaned webpage content. Here, W denotes the complete processed web text and each w_i is an individual segment. The purpose of this equation is to divide the extracted web content into smaller, manageable units, enabling efficient processing and improving retrieval accuracy.

Each segment is converted into vector form for later retrieval:

$$V_w = \text{Embed}(w_i) \quad (3.8)$$

Equation (3.8), $V_w = \text{Embed}(w_i)$, defines the embedding of each web content segment. Here, w_i is a text segment and V_w is its vector representation. The function $\text{Embed}()$

converts the text into a numerical vector capturing semantic meaning. The purpose of this equation is to transform web content into embeddings that support similarity-based retrieval and query processing.

Output:

The output is a processed representation of web content that can be queried by the user through the system interface.

3.3.4 Predictive Analytics Algorithm

The predictive analytics algorithm is used in the IPLA 2.0 phase of the project to classify students into different performance categories based on activity and academic data.

Algorithm:

Input:

The input consists of a feature matrix X derived from student-related data and a corresponding label set y .

Method:

A Random Forest model is trained to learn patterns in the input features and predict the output class:

$$\hat{y} = f(X) \quad (3.9)$$

Equation (3.9), $\hat{y} = f(X)$, represents the prediction generated by the trained model. Here, X is the input feature matrix containing student-related data, f is the trained classifier (e.g., Random Forest), and $\hat{y}$ is the predicted output label. The purpose of this equation is to map input features to a predicted student performance category using the learned patterns from the training data.

Where f represents the trained classifier.

For comparison, Logistic Regression and Random Forest may both be used:

$$\hat{y}_{lr} = f_{lr}(X), \hat{y}_{rf} = f_{rf}(X) \quad (3.10)$$

Equation (3.10), $\hat{y}_{lr} = f_{lr}(X), \hat{y}_{rf} = f_{rf}(X)$, represents predictions from two different models. Here, f_{lr} denotes the Logistic Regression model, f_{rf} denotes the Random Forest model, and $\hat{y}_{lr}, \hat{y}_{rf}$ are their respective predicted outputs. The purpose of this equation is to compare the performance of different classifiers in predicting student outcomes.

Output:

The output is the predicted class label representing student performance or academic risk category.

3.3.5 Multilingual Query Processing Algorithm

The multilingual query processing algorithm enables the system to handle user queries written in different languages.

Algorithm:**Input:**

The input is a user query q written in an arbitrary language.

Method:

The system first detects the language of the input query:

$$\text{lang} = L(q) \quad (3.11)$$

Equation (3.11), $\text{lang} = L(q)$, represents the language detection step. Here, q is the input user query, $L(\cdot)$ is the language detection function, and lang is the identified language of the query. The purpose of this equation is to determine the language in which the query is written so that appropriate processing or translation can be applied.

If required, the query is translated into English for internal processing:

$$q' = T(q, \text{lang}, \text{English}) \quad (3.12)$$

Equation (3.12), $q' = T(q, \text{lang}, \text{English})$, defines the translation process. Here, q is the original query, lang is the detected source language, $T(\cdot)$ is the translation function, and q' is the translated query in English. The purpose of this equation is to convert the query into a common processing language, ensuring compatibility with the retrieval and response generation pipeline.

The translated query is then passed through the retrieval-based response generation pipeline.

Output:

The output is a meaningful response generated for the user query while preserving its original meaning across languages.

3.3.6 Entity Extraction Algorithm

The entity extraction algorithm forms the foundation of the schema design agent. Its purpose is to identify the important domain objects present in the natural-language requirement description.

Algorithm:**Input:**

The input consists of a natural-language requirement description X .

Method:

The system analyzes the text using linguistic and contextual cues to identify candidate entities. The extraction process is represented as:

$$E = f(X) \quad (3.13)$$

Equation (3.13), $E = f(X)$, represents the entity extraction process. Here, X is the input natural-language requirement description, $f(\cdot)$ is the text analysis and extraction function, and E is the set of extracted entities. The purpose of this equation is to identify and extract meaningful domain objects from unstructured text, which serve as the foundational components for schema design.

Where E is the set of extracted entities and f represents the text analysis and extraction function.

The system filters irrelevant terms and retains meaningful domain objects as candidate schema components.

Output:

The output is a structured set of extracted entities $E = \{e_1, e_2, \dots, e_n\}$, which forms the basis for later schema generation.

3.3.7 Relationship Detection Algorithm

The relationship detection algorithm identifies associations between the extracted entities and determines the type of connection among them.

Algorithm:

Input:

The input consists of the entity set E and the original requirement description.

Method:

The system examines action-based patterns and contextual dependencies in the text to form a relationship set:

$$R = \{(e_i, e_j, \text{type})\} \quad (3.14)$$

Equation (3.14), $R = \{(e_i, e_j, \text{"type"})\}$, represents the relationship extraction process. Here, e_i and e_j are entities from the set E , and "**type**" denotes the nature of the relationship between them (such as one-to-one, one-to-many, or many-to-many). The purpose of this equation is to define how extracted entities are connected based on contextual and action-based patterns in the requirement description, forming a structured representation of relationships.

Where type may represent one-to-one, one-to-many, or many-to-many relationships.

Output:

The output is a set of detected relationships connecting the extracted entities.

3.3.8 Schema Generation Algorithm

The schema generation algorithm converts the extracted entities and detected relationships into relational database structures.

Algorithm:

Input:

The input consists of the entity set E and relationship set R .

Method:

Each entity is mapped to a relational table. If T_e represents the table generated from entity e , then:

$$T_e = \{\text{id}, A_e\} \quad (3.15)$$

Equation (3.15), $T_e = \{\text{"id"}, A_e\}$, represents the mapping of an entity to a relational table. Here, e is an entity from the set E , T_e is the corresponding table, "**id**" is the primary key, and A_e is the set of attributes associated with that entity. The purpose of this equation is to define how each extracted entity is structured into a table with a unique identifier and its relevant attributes.

Where A_e is the attribute set associated with entity e .

For many-to-many relationships, a junction table is introduced:

$$T_{ij} = \{\text{id}_i, \text{id}_j, A_{ij}\} \quad (3.16)$$

Equation (3.16), $T_{ij} = \{\text{"id"}_i, \text{"id"}_j, A_{ij}\}$, represents the creation of a junction table for many-to-many relationships. Here, " id_i " and " id_j " are foreign keys referencing the related entity tables, and A_{ij} denotes any additional attributes of the relationship. The purpose of this equation is to resolve many-to-many relationships by introducing an intermediate table that maintains proper relational structure.

Output:

The output is a complete relational schema containing tables, attributes, keys, and relationship mappings.

3.3.9 ER Diagram Generation Algorithm

The ER diagram generation algorithm converts the schema structure into a visual graph-based representation.

Algorithm:

Input:

The input consists of the entity set E and relationship set R .

Method:

The schema is represented as a graph:

$$G = (V, E_g) \quad (3.17)$$

Equation (3.17), $G = (V, E_g)$, represents the graph-based structure used for ER diagram generation. Here, G is the overall graph, V is the set of nodes corresponding to entities from E , and E_g is the set of edges representing relationships from R . The purpose of this equation is to model the relational schema as a graph, where entities are visualized as nodes and relationships as labeled connections, enabling intuitive understanding of the database structure.

Where V denotes the set of entity nodes and E_g denotes the relationship edges. Each entity is represented as a node, and each relationship is rendered as an edge with the appropriate label and direction.

Output:

The output is an ER-style diagram visually representing the schema structure.

3.3.10 Dataset-Based Validation Algorithm

The validation algorithm evaluates the correctness of the generated schema by comparing extracted entities against expected entities from curated examples.

Algorithm:

Input:

The input consists of the expected entity set E_{exp} and the generated entity set E_{gen} .

Method:

The correctly identified entities are computed as:

$$E_{correct} = E_{exp} \cap E_{gen} \quad (3.18)$$

Equation (3.18), $E_{correct} = E_{exp} \cap E_{gen}$, represents the set of correctly identified entities. Here, E_{exp} is the expected entity set, E_{gen} is the generated entity set, and $E_{correct}$ contains the entities common to both sets. The purpose of this equation is to identify the overlap between expected and predicted entities, indicating correctly extracted results.

The extraction accuracy is calculated as:

$$\text{Accuracy} = \frac{|E_{correct}|}{|E_{exp}|} \times 100 \quad (3.19)$$

Equation (3.19), $\text{Accuracy} = \frac{|E_{\text{correct}}|}{|E_{\text{exp}}|} \times 100$, defines the accuracy of the entity extraction process. Here, $|E_{\text{correct}}|$ is the number of correctly identified entities and $|E_{\text{exp}}|$ is the total number of expected entities. The purpose of this equation is to measure how effectively the system extracts relevant entities by calculating the percentage of correct predictions.

Output:

The output is an accuracy value indicating how effectively the system identifies the expected entities from the requirement description.

This is the correct algorithm flow for your report. Do not use medical-image algorithms like EfficientNetB7 or EfficientNetB3 in this section, because they do not match your project topic.

3.4 Description of Datasets, Requirements and Tools :-

The system must have a systematic set of natural-language software requirements and their anticipated schema-level outputs to develop and test the proposed AI-powered automated database schema design agent effectively. These data are not image data or sensor data, but text data as the project is aimed at processing software problem statements and transforming them into relational database design artifacts. The data set of this project is a view of curated requirement descriptions across various application areas like hospital management, e-commerce, library systems, banking, hotel booking, school administration and inventory management. Every requirement description is a real-life software scenario and is accompanied with the anticipated entities that are used as reference outputs to validate them.

The primary intention of the dataset is to subject the system to a vast pool of domain patterns and style of requirement writing. Because software requirements can vary in vocabulary, structure and complexity, multi-domain examples aid in the more efficient handling of various problem statements by the system. The dataset can also be evaluated since it allows one to determine the generated entities against the expected entities. This allows the performance of the system to be measured in a systematic way and to ensure the accuracy of the system in recognising the important schema components based on the textual input.

Besides manually typed input, the dataset can also be inputted in dataset mode, where predefined examples can be selected and automatically processed. This renders the system applicable in test-retest, experimentation, and demonstration. Reproducibility is also favored by the curated nature of the dataset, as the same examples can be applied in many runs to see how the system behaves and improves with time.

Hardware Requirements

The proposed system needs a moderate computing environment to handle software requirement descriptions, produce schema output, render ER-style drawings, and verify output. In contrast to image-based deep learning systems, this project does not rely on intensive training on heavy GPUs, but it still needs enough computing power to facilitate easy backend processing, data processing, text analysis, and visualization.

A modernized dual core or quad core processor can execute the pipeline effectively in a system. Because the project is primarily a text-based application, CPU-based computation suffices to handle most tasks including requirement preprocessing, entity extraction, relationship detection, SQL generation and evaluation. In the case of multiple test cases being run or repeated analysis being done in the course of an experiment, a faster processor is more responsive though.

At least 8 GB of RAM is advised to ensure that the system does not run out of memory when loading datasets, intermediate processing, and backend services, as well as visualizing these datasets. A system with 12GB and above memory is more desirable to have this system run smoothly and also to multitask. A fast storage like SSD is also advantageous as it enhances the speed of accessing files particularly when dealing with datasets, generated outputs, diagrams, as well as repetitive testing processes.

In general, hardware needs of the system are within the scope of typical academic and prototype computing systems. This renders the solution suggested practical with personal systems, student laptops and institutional lab installations without high-performance infrastructure specialization.

Software Requirements

The main programming language to implement the proposed system is Python due to its simplicity, flexibility, as well as rich ecosystem of libraries to perform natural language processing, back-end development, structured data processing and visualization. Coding environments like Jupyter Notebook, local Python environments, or regular IDEs support the development of the system and offer a convenient platform to test, debug, and integrate the system.

The system is implemented on the backend with the help of FastAPI as a lightweight and efficient request-handling framework and an orchestration of processing modules. FastAPI assists in the linkage between the input of the user and processing elements and the creation of outputs like extracted entities, SQL schema, diagrams, and evaluation outcomes in an organized responsive way.

In frontend interaction, we will employ Streamlit to give a user-friendly interface where they can input software requirement descriptions and can view outputs. This simplifies the system to both the technical and non-technical users. The interface has been provided with manual mode and dataset mode thus making the application more practical.

The visualization tool is based on Graphviz to create ER-style schema diagrams. It facilitates the graphical representation of the nodes and the relationships in a comprehensible form, and the generated schema can be more easily comprehended and proven. Storage is also needed in CSV format to store the datasets entries and the anticipated outputs that are to be used in validation.

The software environment is thus founded on Python, FastAPI, Streamlit, Graphviz, and CSV-based data processing assistance. Combining all these technologies, it becomes possible to create a complete end-to-end schema design automation workflow.

Libraries and Tools

The system uses a number of libraries to support different aspects of the project.

Pandas:

Pandas is applicable in working with structured data like requirement datasets, anticipated entity lists and validation records. It makes reading, storing, filtering, and comparing the entries of a dataset easy and it is an essential part of the evaluation support.

NumPy:

NumPy is applicable to both numerical operations and internal data processing wherever mathematical processing or structured array representation is needed in the workflow.

re (Regular Expressions):

Text cleaning, tokenization and pattern based extraction of the input requirement descriptions are done by the regular expression library. It has many applications in text processing, especially finding meaningful words and de-noising textual input.

FastAPI:

The back-end application FastAPI is used to control request streams, coordinate the modules of the system, and communicate between the processing pipeline and the frontend.

Streamlit:

Streamlit offers the user interface that requires natural-language requirement descriptions and provides the selection of examples of datasets and visualizes the output generated in an interactive manner.

Graphviz:

ER-style schema visualization is done using Graphviz. It assists in generating diagrams that depict extracted entities and relationships, simplifying the generated schema to understand.

Matplotlib:

Matplotlib can be utilized in displaying evaluation summations or result graphs as needed when analyzing and recording.

CSV Handling Tools:

Input requirements and anticipated entities are stored and managed through CSV-based support to verify them. This renders the process of evaluation systematic and repeatable.

General Python Utilities:

String processing, file processing, XML processing or output formatting, and integration tasks needed by the entire workflow are handled by various built-in Python modules.

Visualization and Development Tools

Local Python environments and notebook-based tools are used to develop, test, and experiment with models. These tools facilitate testing of in-between results, debugging extraction rules, verifying schema generating, and comparing among various examples of requirements. They are also useful in recording the workflow in an understandable manner when developing the project.

Graphviz is the primary visualization tool in the system, which can be used to automatically create schema diagrams based on the extracted entities and relationships. These graphic deliverables of the project are valuable both in technical validation and documentation and presentation. Moreover, text editors, notebook applications, and

local development tools facilitate easy integration of the backend code, interface design, and testing modules.

The system aids in the storage of the generated outputs like SQL scripts, extracted entities, validation results, and document-ready content. This enhances reproducibility and assists in keeping a complete development record.

Workflow Support and System Integration.

The above datasets together with libraries, tools, and frameworks can be combined to enable the system to work as a complete end-to-end workflow. The workflow starts with the requirement acquisition followed by text preprocessing, entity extraction, relationship detection, schema synthesis, SQL generation, ER-diagram rendering, and validation. The software components of each stage are supported, and contribute to modularity, making the system scalable and maintainable.

Backends, data manipulation and visualization frameworks can be integrated to enable the system to read and process textual inputs without stuttering and produce various outputs. FastAPI and Streamlit make sure that the interface layer does not lose contact with the processing modules inside, whereas Graphviz and Pandas facilitate visualization and assessment, respectively.

As the system is designed in a modular manner, every component can work on its own and yet contribute to the overall working process. It is significant to future enhancements, as single components like entity extraction, relationship detection, or SQL generation can be enhanced without modifying the entire architecture. This kind of integration not only makes the project functional but also flexible to be expanded in the future.

Overall Significance

The selection of appropriate datasets, hardware, software and supporting libraries play a key role in ensuring the success of the proposed system. The structured environment ensures that the system is able to handle requirement descriptions, generate useful database schema output and offer reliable validation results. The chosen frameworks

and tools allow the implementation to be practical, accessible, and can be used in academics and also at a prototype level.

All in all, the selected requirements and tools offer a solid base to develop a scalable, efficient, and reliable AI-driven automated database schema design system. They facilitate the entire process of natural-language requirement understanding through formal schema generation and render the system usable as a technical solution and a design assistant helpful in learning.

CHAPTER-4

Results and Observations

4.1 Stepwise description of Results

The outcomes of the general work are outlined in stages, starting with the Intelligent Personal Learning Assistant, going on to the improved IPLA 2.0 system, and lastly to the AI-Powered Automated Database Schema Design Agent. The stages are all marked by clear advancements in the capabilities of the system and are an indication of the gradual advancement of the project as a project based on academic support to intelligent software design automation.

4.1.1 Intelligent Personal Learning Assistant (IPLA)

The initial stage of the project was to develop an Intelligent Personal Learning Assistant, an assistant that can assist students in document-based answering questions, web content analysis, and interactive academic answering. The findings of the given stage prove that the system manages to combine AI techniques based on retrieval and a simple and convenient interface.

The login interface of the system is shown in Figure 4.1. This page will serve as the gateway to the user and it will facilitate a secure authentication process that will be followed before the user can access the internal modules. Based on the figure, we can see that the interface is easy-to-use, well-structured, and able to be used in academics.

The fact that the login module was successfully implemented proves that the system can provide controlled access and role-based interaction.

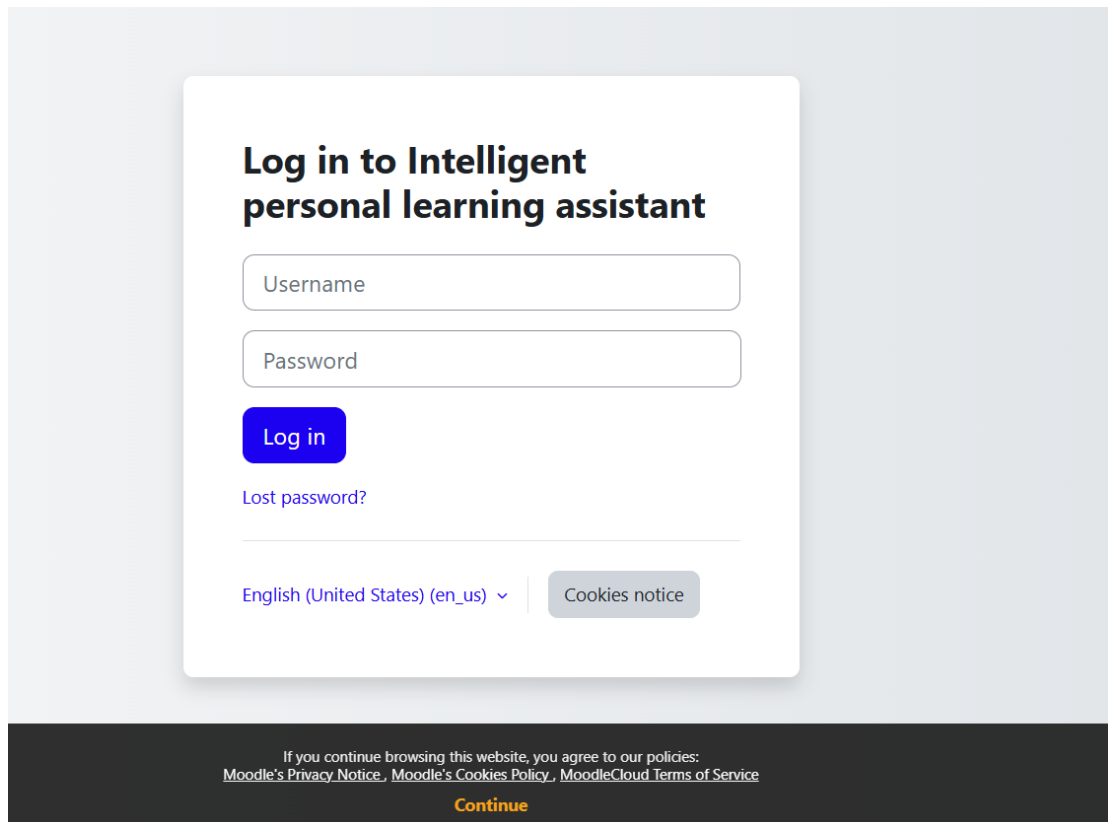
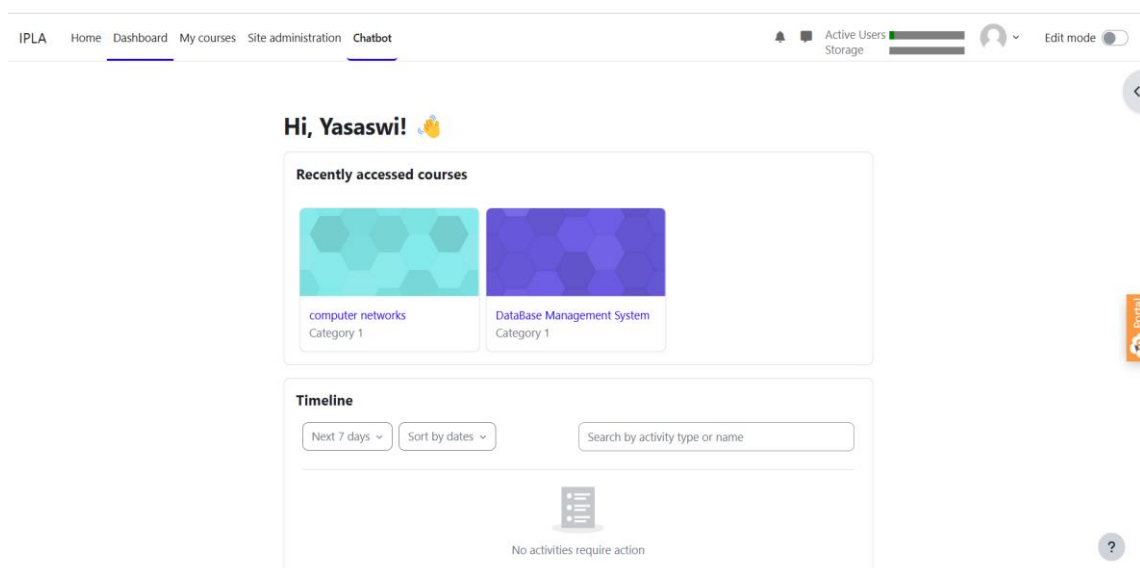


Figure 4.1: Login Page Interface

Figure 4.2 shows the administrative interface. This module allows centralized control of the system and illustrates that the platform is not restricted to working with students only. The interface is also friendly to monitor and manage the content, which



implies that the system has been developed as a full-fledged educational platform and not just as a single chatbot.

Figure 4.2: IPLA Admin Interface

The main AI chatbot interface is shown in Figure 4.3. This number illustrates how question answering and retrieval based academic support can be integrated. The interface presented demonstrates that the users can engage with the system in an orderly manner and receive pertinent academic responses. This outcome validates the fact that the chatbot module can process user queries and offer relevant outputs because of its user-friendly design.

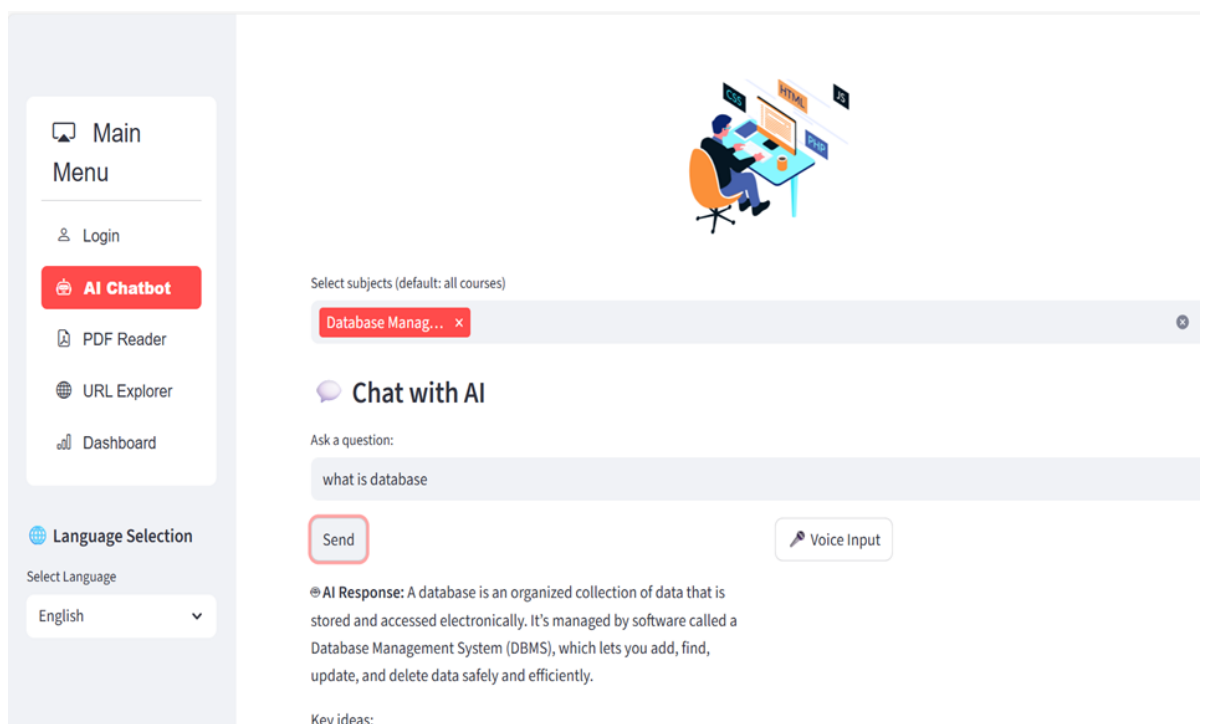


Figure 4.3: IPLA AI Chatbot Interface

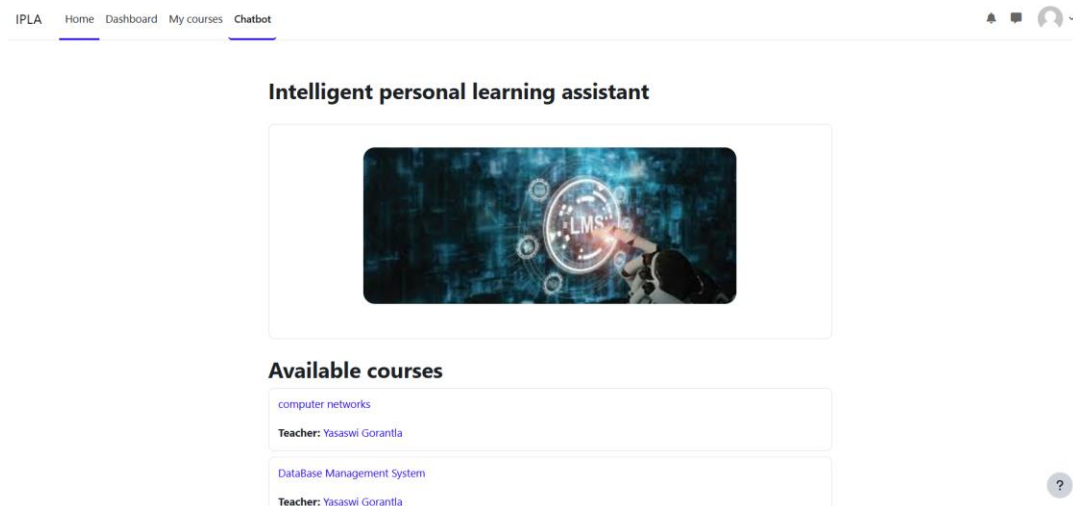


Figure 4.4: Student Assistant Interface

The student assistant home interface is shown in Figure 4.4. This module shows the conversational support provided by the system to academic interaction. Based on the figure, it can be observed that the assistant has been developed to enhance ease of use and simplify the overall interaction process by students. The outcome proves that the academic support level of the system has been introduced in a convenient and useful manner.

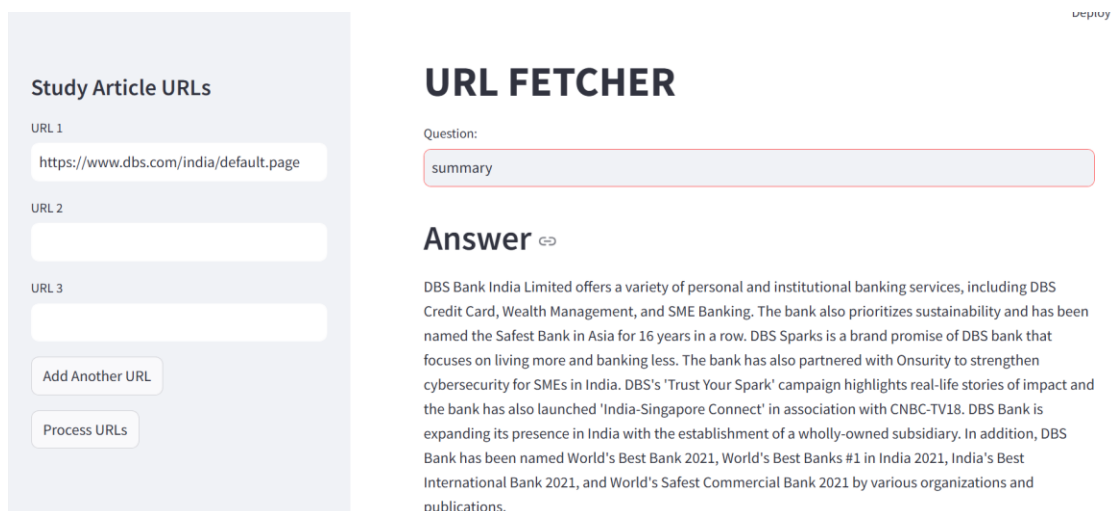


Figure 4.5: URL-Based Content Processing

The URL-based content processing module is shown in Figure 4.5. This finding is important in that it demonstrates that the system is capable of retrieving, washing, and processing information located on external web. The figure shows that URL content can be converted to a usable source of knowledge to answer a query. This goes a step further and expands the system to uploaded files and makes it more useful in more practical academic settings.

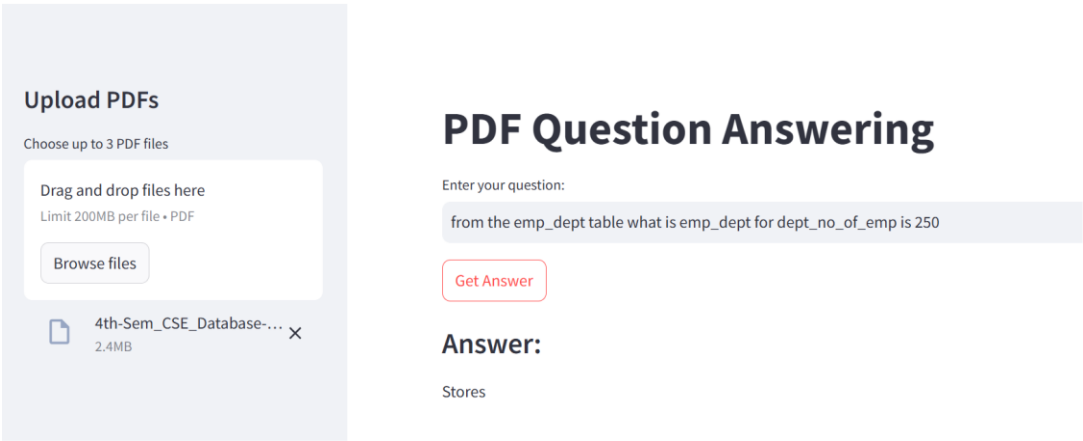


Figure 4.6: PDF Processor

The PDF processor module is presented in Figure 4.6. This finding indicates that PDF documents uploaded can be transformed into searchable and queryable materials. Based on the figure, one can see that the system manages to extract the information in the form of the static academic material and enables the users to engage with it by using the help of AI. This confirms the effectiveness of the document-processing workflow.

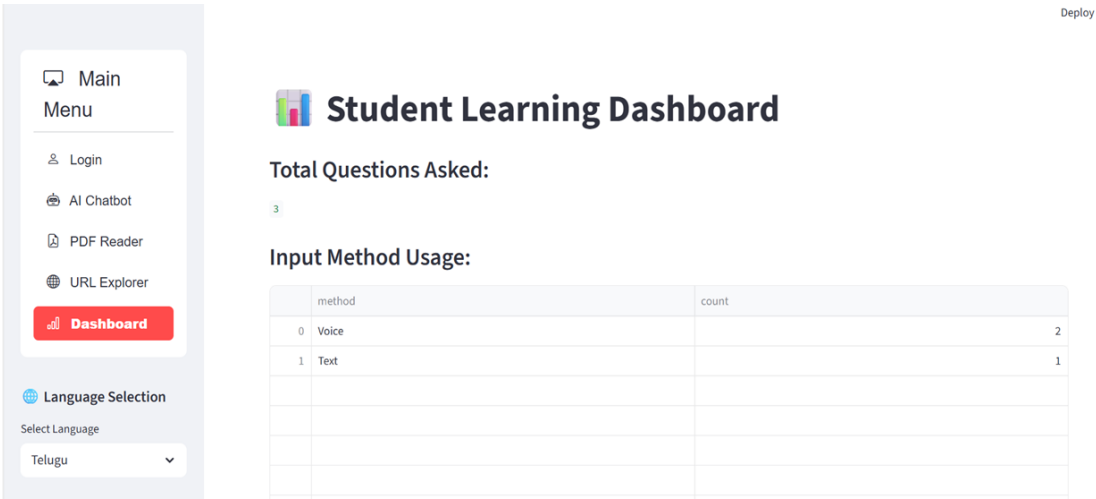


Figure 4.7: Learning Assistant Dashboard

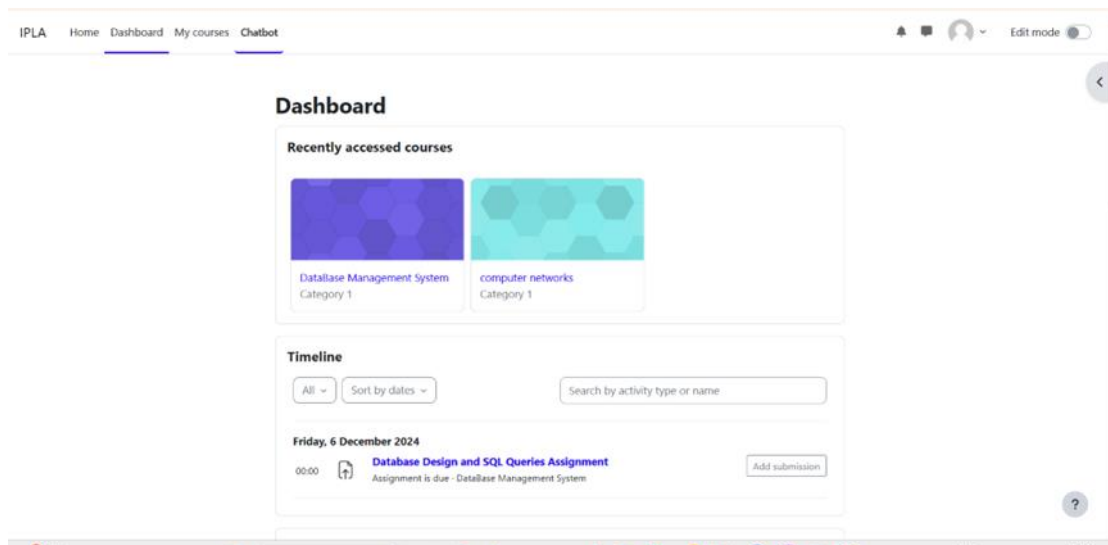


Figure 4.8: Courses and Materials View

The dashboard interface is shown in Figure 4.7, while the courses and materials view is shown in Figure 4.8. These findings reveal that the system facilitates structured academic engagement by displaying the course information, the materials accessed recently and student activity in an organized manner. Combining this data, it can be stated that the initial stage of the project will offer not only AI-powered support but also a simplistic learning management style setting.

In general, the findings of the initial stage demonstrate that the IPLA system effectively integrates the control of logins, the management of content, the support of chatbots, the processing of documents, the web-based search, and the interaction with the dashboard into a single academic support system.

4.1.2 IPLA 2.0 – Multilingual, Predictive and Adaptive Learning Assistant

The second stage of the project builds on the previous learning assistant by adding multilingual support, better response generation based on retrieval, predictive analytics, and better student interaction capabilities. The outcomes of this stage indicate that the system will go past mere academic support and start facilitating adaptive learning and data-driven decision-making.

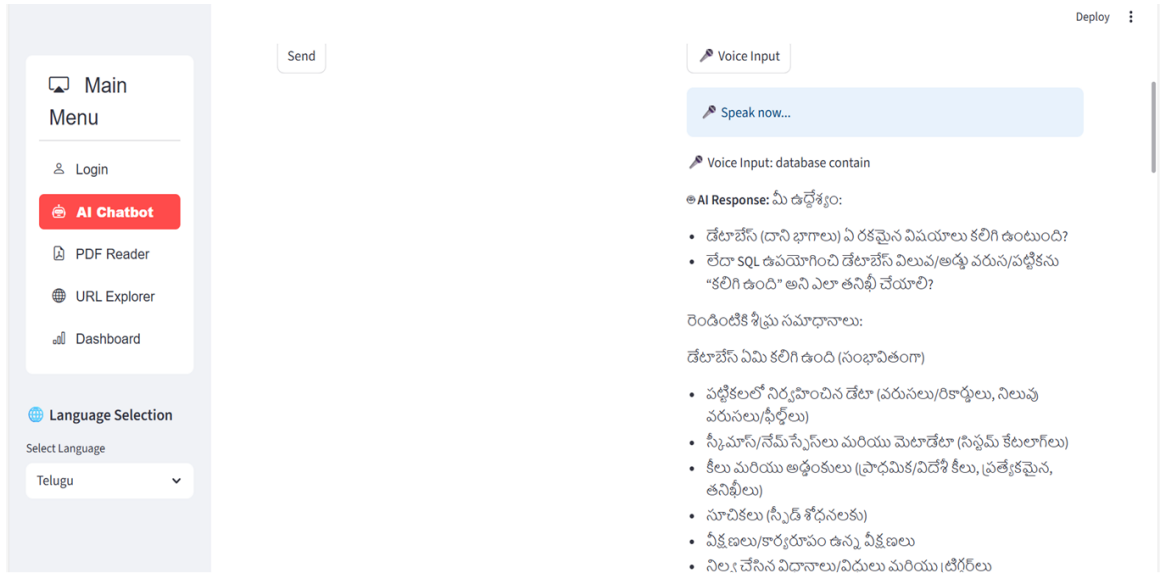


Figure 4.9: Multilingual Interaction

Figure 4.9 demonstrates the multilingual interaction ability. Based on this figure, it is possible to note that the system is able to handle the user queries in the regional or alternate language and still give meaningful response. This finding proves that multilingual module increases the access and makes the platform more helpful to a broader group of students.

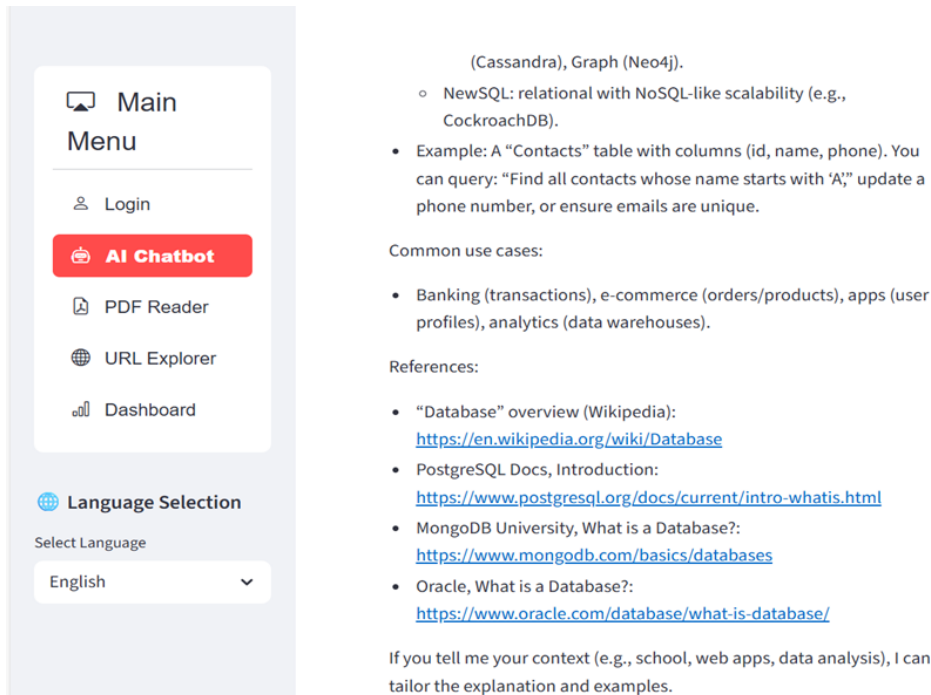


Figure 4.10: RAG-Based Grounded Academic Response

The retrieval-augmented response generation module is illustrated in Figure 4.10. The figure shows that the system produces answers based on the content they have retrieved and not by generically generating answers. The outcome is that the responses are always strongly related to the query of the user and source of knowledge available, which enhances accuracy and context relevance.

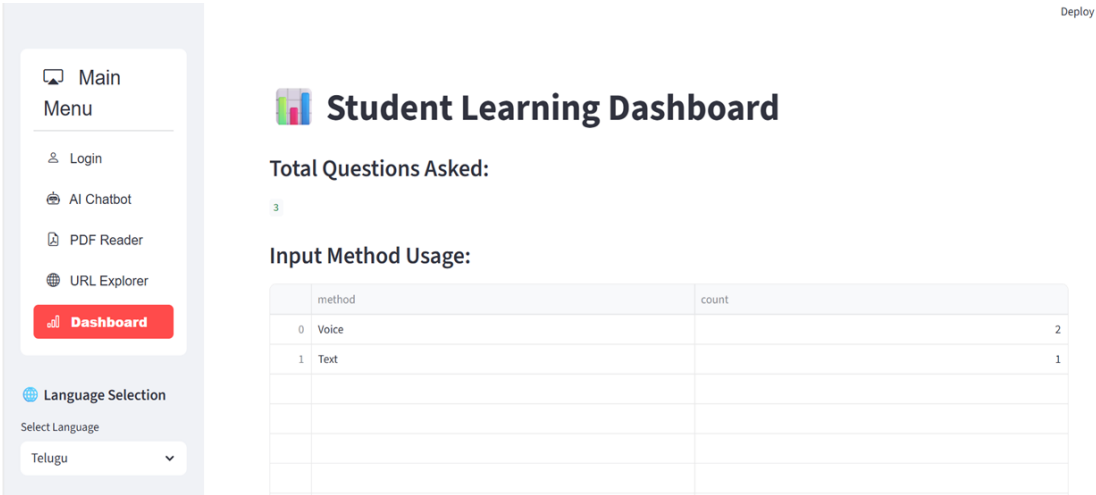


Figure 4.11: Student Learning Dashboard

The student learning dashboard of the enhanced system is shown in Figure 4.11. This finding is indicative of the better streamlining of student activity, system outputs, and flow of interaction. The dashboard affirms that IPLA 2.0 offers a more organized and flexible learning experience as compared to the initial phase.

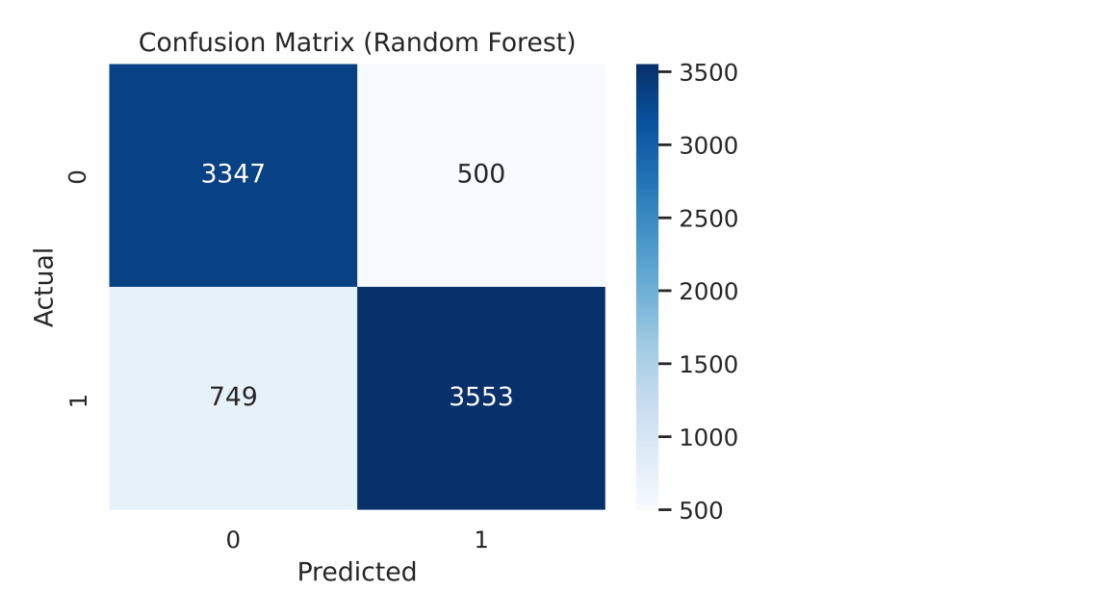


Figure 4.12: Confusion Matrix — Random Forest Classifier(IPLA-2.0)

The confusion matrix in Figure 4.12 indicates the predictive analytics output. The figure shows that the classification model works well in distinguishing between the categories of students with the majority of predictions being along the diagonal. This shows that the predictive module is able to classify academic performance data with consistency and reliability.

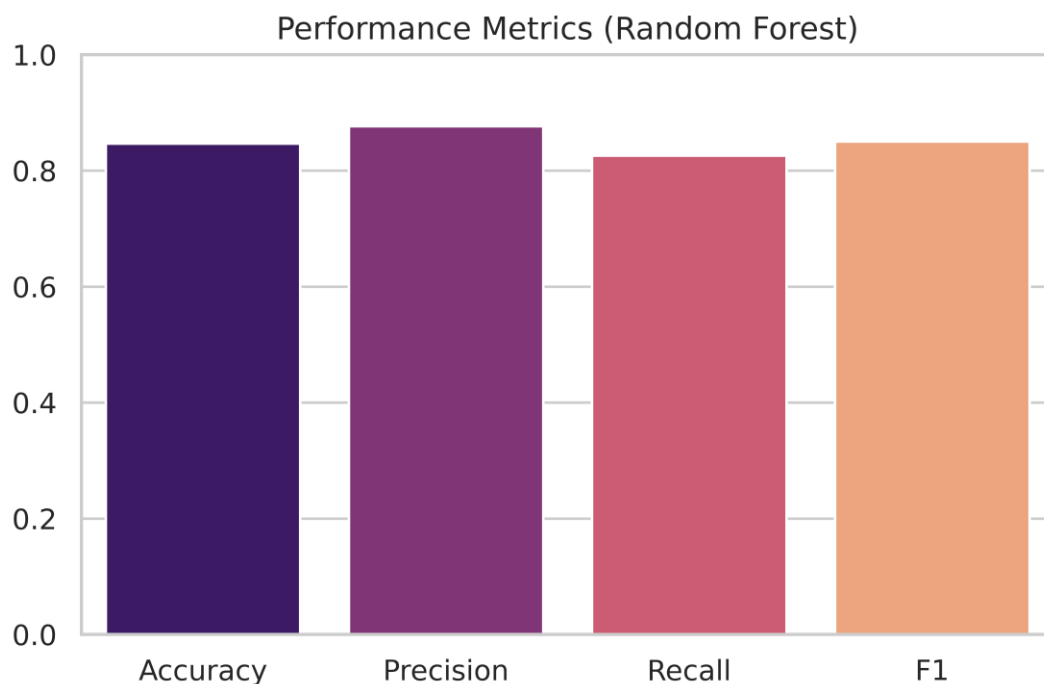


Figure 4.13: Model Performance Metrics Comparison — IPLA 2.0 Predictive Analytics

Table 4.1 and Figure 4.13 also provide further evidence of the comparative performance of the models. Random Forest model is more accurate, recalls more, and has higher F1-score when compared to the Logistic Regression. This proves that the predictive analytics aspect of IPLA 2.0 is not a mere functionality, but an aspect that can be supported by quantifiable outcomes. The graph also depicts that there is a balance of performance in terms of evaluation metrics, which means that the classification is stable.

On balance, the second phase results are that IPLA 2.0 effectively builds upon the previous academic support system that includes multilingual interaction, ground

response generation, dashboard-based learning support, and predictive analytics. These features make the system smarter, dynamic, and applicable to real-world education.

4.1.3 AI-Powered Automated Database Schema Design Agent

The last stage of the project is automated database schema generators, which are based on natural-language software requirements descriptions. This is the greatest contribution of the project and is the shift of educational assistance to intelligent software design automation. The outputs of this step indicate that the system is capable of analyzing software needs, determining schema elements, creating relational frameworks, and displaying them in a text and visual format.

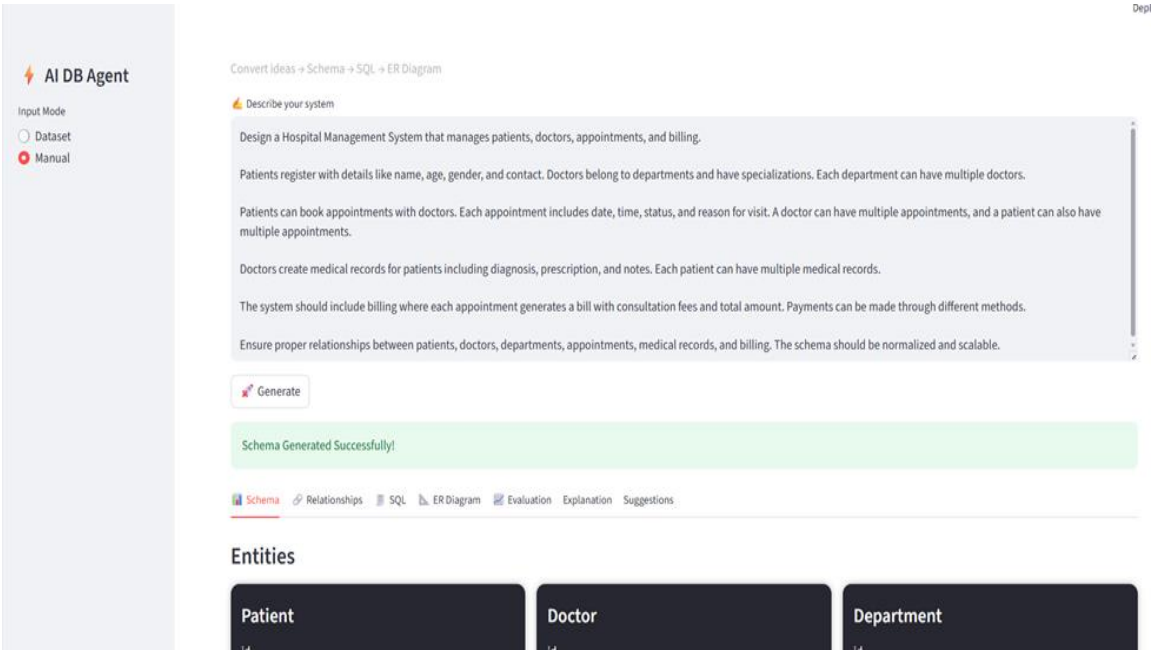


Figure 4.14: Schema Generation Interface — NL Input to SQL Output

The main schema generation interface is shown in Figure 4.14. This number shows that the system takes a natural-language description of requirement as an input and processes it through several steps. The fact that there are tabs, or sections, that allow the user to view a summary of the schema, extract entities, map relationships, generate SQL, and analyze it shows that the system is based on a full pipeline rather than a single, distinct operation. This attests to the practicality of the interface.

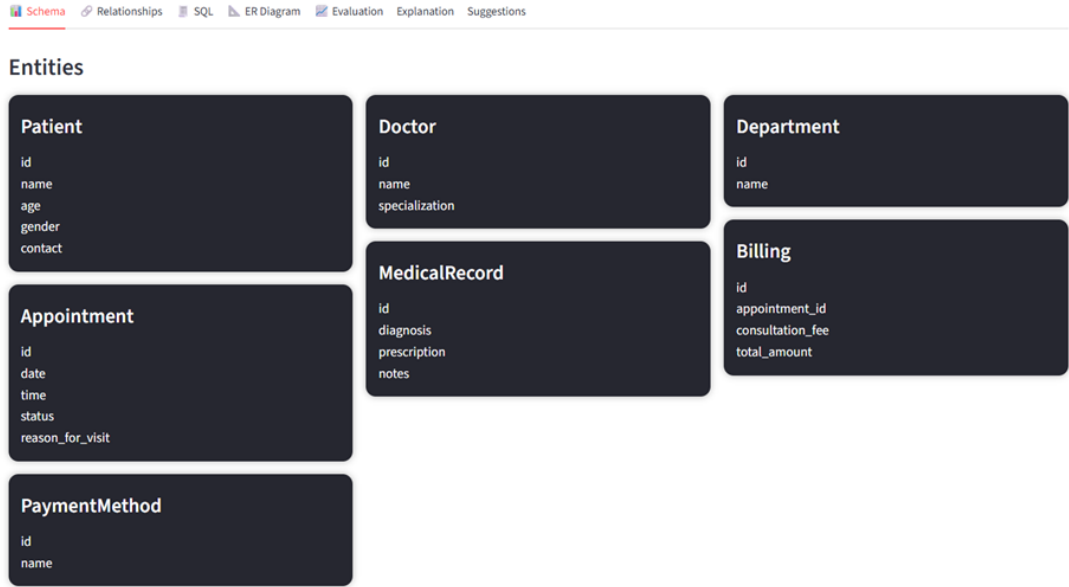


Figure 4.15: Extracted Entities and Attributes

The extracted entities and their associated attributes are shown in Figure 4.15. In the figure, one can note that the system recognizes the key domain objects based on the description of the requirements and arranges them hierarchically. The importance of the result is that it demonstrates the entity extraction part is able to extract significant information on a schema level on natural-language input.

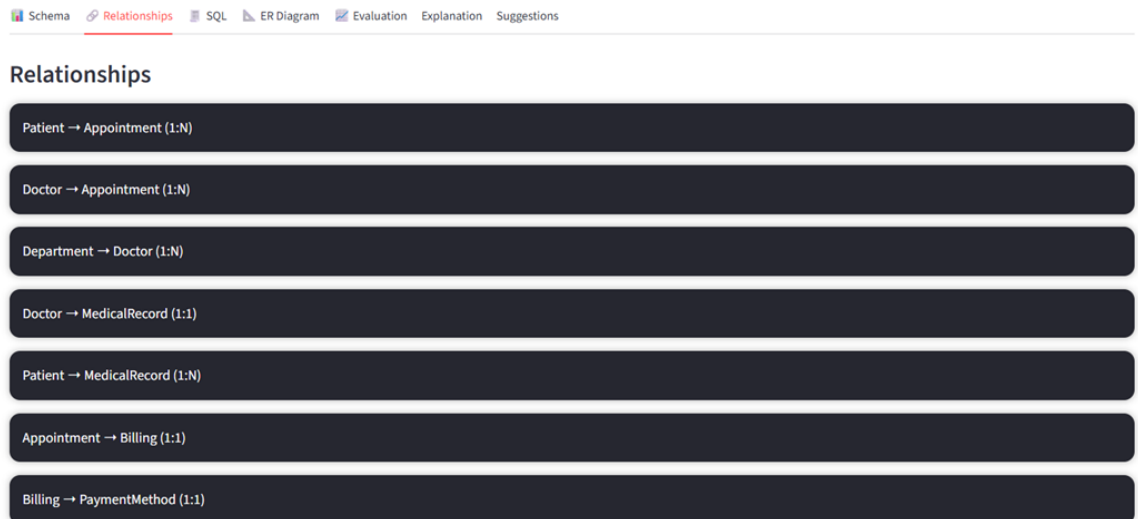


Figure 4.16: Relationship Detection Output

The relationship detection output is presented in Figure 4.16. This number indicates that the system can determine relationships of the extracted entities in the problem

domain. The existence of organized relations proves the fact that the system is not limited to the simple keyword extraction but tries to represent the logical relations between the entities. It constitutes a required step towards the generation of a significant relational schema.

```
CREATE TABLE Patient (
  id INT PRIMARY KEY,
  name VARCHAR(255),
  age VARCHAR(255),
  gender VARCHAR(255),
  contact VARCHAR(255)
);

CREATE TABLE Doctor (
  id INT PRIMARY KEY,
  name VARCHAR(255),
  specialization VARCHAR(255)
);

CREATE TABLE Department (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);

CREATE TABLE Appointment (
  id INT PRIMARY KEY,
  date VARCHAR(255),
  time VARCHAR(255),
  status VARCHAR(255),
  reason_for_visit VARCHAR(255)
);

id INT PRIMARY KEY,
date VARCHAR(255),
time VARCHAR(255),
status VARCHAR(255),
reason_for_visit VARCHAR(255)
);

CREATE TABLE MedicalRecord (
  id INT PRIMARY KEY,
  diagnosis VARCHAR(255),
  prescription VARCHAR(255),
  notes VARCHAR(255)
);

CREATE TABLE Billing (
  id INT PRIMARY KEY,
  appointment_id VARCHAR(255),
  consultation_fee VARCHAR(255),
  total_amount VARCHAR(255)
);

CREATE TABLE PaymentMethod (
  id INT PRIMARY KEY,
  name VARCHAR(255)
);
```

Download SQL

Figure 4.17: SQL DDL Generation

The SQL DDL statements generated are presented in Figure 4.17. This output shows that the schema extracted has been converted to implementation-ready database code. The figure validates that the system is able to produce relational table definitions and key constraints and the output is not only useful in the conceptual design but also in the actual creation of the database.

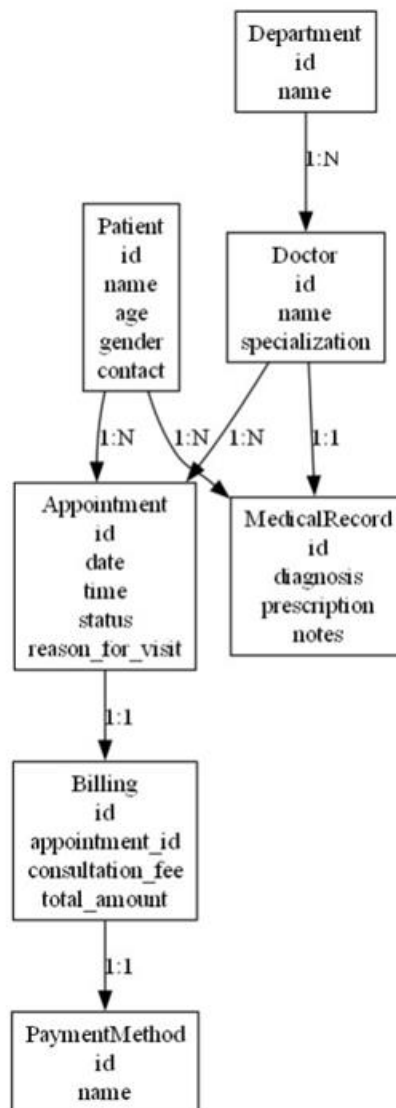
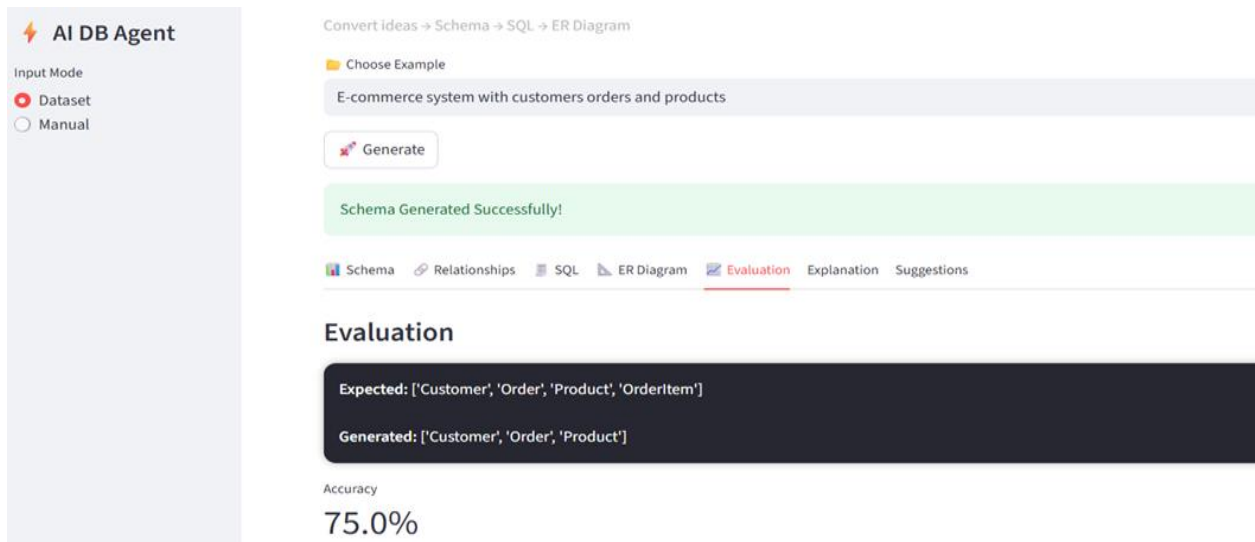


Figure 4.18: ER Diagram Generated by the Schema Design Agent

Figure 4.18 shows the output of the ER-style diagram. This number is a graphical representation of the system generated entities and relations. The significance of the result is that it gives an interpretable and checkable version of the schema, enabling the user to check whether the design generated represents the desired system structure.



The output of the validation is presented in Figure 4.19. This number shows that the system will match generated entities to expected entities and calculate an accuracy output. In the provided example, the system will have an entity extraction accuracy of 75 percent, which demonstrates that the system will find most of the significant domain objects in the requirement description. This affirms that the schema design agent does not just produce outputs but also facilitates organized analysis of its performance.

Overall, the results of the final phase show that the proposed AI-powered automated database schema design agent successfully transforms natural-language software requirements into database design artifacts such as entities, relationships, SQL schema, ER diagrams, and evaluation outputs. This makes the final system a complete and practical tool for early-stage schema generation.

4.2 Result Analysis and Test Case Results

A sequence of increasing capability of the system is evident in the results of the various phases of the project, starting with retrieval-based academic support, then multilingual and predictive support, and ultimately automated database schema generation. This evolution suggests that the project has transformed to managing educational interaction to resolve a more formal software design issue. The last system shows that artificial intelligence can be utilized not only to support academic tasks but also to create meaningful database design artifacts based on natural-language software requirements.

The Intelligent Personal Learning Assistant generated consistent and significant outcomes in academic query processing in the initial stage of the project. The interface of the system, i.e. the interface of the system where the user logs in as it is seen in Figure 4.1, confirms that the system allows the user to interact with the system in a secure and well-organized manner. The administrative interface of Figure 4.2 shows that the platform is to be built on management and control features as opposed to being a single-user tool. As Figure 4.3 indicates, the chatbot interface can receive academic queries and provide corresponding answers. The student assistant home module as depicted in Figure 4.4 also enhances usability and it also goes to show that conversational academic assistance has been introduced in a systematically and practically.

Other outputs of this initial step can be seen in the content processing modules. Content processing outcome in the URL format as in Figure 4.5 is a confirmation that the system is able to retrieve and interpret web-based information to answer questions. Likewise, the result of the PDF processing in Figure 4.6 indicates that stable academic documents can be turned into queryable source of knowledge. The dashboard and course content in Figure 4.7 and Figure 4.8 denote that the system effectively combines document retrieval, web based support, and academic interaction within an entire platform. These findings substantiate that the initial stage of the project works well in retrieval-based educational support.

The second phase IPLA 2.0 built on the previous system by adding multilingual support, better response generation based on retrieval and predictive analytics. The multilingual interaction output as illustrated in Figure 4.9 illustrates that the system is capable of handling queries that are not limited to one language and still provide meaningful output. This validates enhanced access and usability by various users. The recovery-enriched generation of responses in Figure 4.10 also suggests that the responses are based on the retrieved content of relevance, enhancing the contextual accuracy and minimising the generic responses. The result that is presented in Figure 4.11 on the dashboard also confirms the fact that the improved system is more adaptive and structured to provide learning.

The result of the predictive analytics is presented in the confusion matrix Figure 4.12 and graphical metrics comparison Figure 4.13. These findings suggest that the Random

Forest model is more effective than the Logistic Regression with regard to classification performance. The density of predictions around the diagonal of the confusion as well as the metric comparison reveal that the model can be used to predict performance categories with uniformity, whereas the metric comparison illustrates that predicting performance using Random Forest is stronger in terms of recall and F1-score. This segment of the analysis proves that IPLA 2.0 is effective in integrating retrieval support and predictive intelligence and, therefore, the system becomes more adaptive and data-oriented.

Table 4.1 shows the comparative performance of the predictive models employed in IPLA 2.0.

Table 4.1 Performance Comparison — Logistic Regression vs Random Forest (IPLA 2.0)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.832	0.870	0.802	0.835
Random Forest	0.844	0.869	0.830	0.849

As can be seen in Table 4.1, the Random Forest model is more effective than the Logistic Regression at most evaluation measures. The high recall and F1-score denote that Random Forest is more efficient in determining pertinent categories of students and, therefore, is more appropriate in predictive analytics in the better learning assistant system.

The last and the most significant analysis of the results is connected with the AI-Powered Automated Database Schema Design Agent. This step was verified with the help of natural-language software requirement description in areas like hospital management. This stage is primarily represented as the main system interface as illustrated in Figure 4.14, wherein requirement description is handled by various output modules. The figure affirms that the system adheres to end-to-end workflow as opposed to carrying out one-off task. The fact that there are distinct outputs of entities, relationships, SQL schema, and evaluation means that the design pipeline is integrated.

The entity extraction is the first significant output of this step and presented in Figure 4.15. Based on this figure, it is evident that the system is effective in extracting the core domain components out of the input requirement description and the system will arrange them in a structured form. This is a confirmation that the system can translate natural-language formulations of problems into meaningful schema-level concepts. The extracted entities consist of attribute groupings as well, demonstrating that the resulting output is more than simple listing of keywords

The second significant output is the relationship detection output in Figure 4.16. This figure illustrates the fact that the system recognizes logical links between the extracted entities and expresses them in a tabular form. This outcome matters since the design of databases does not just involve the identification of entities, but also correctly interpreting database dependencies among the entities. The relationship output thus indicates that the system does not merely extract on a surface level but does structural analysis.

The SQL generation result is shown in Figure 4.17. This figure shows that the conceptual schema derived out of the requirement description is effectively translated into SQL Data Definition Language statements. The output produced is the relational table structures and key mappings, which implies that the system can produce implementation-oriented artifacts with natural-language input. This proves that the system is not only conducive to the practical schema development but also conceptual interpretation.

Another significant outcome is the ER-style visual representation in Figure 4.18. The database design that is generated becomes easier to comprehend and prove with the help of the visual output that is provided to display the extracted entities and relationships in diagram format. This finding enhances the system interpretability and facilitates academic learning and design review. It also assures that the system is consistent in its textual outputs and visual representations.

The validation outcome in Figure 4.19 is quantifiable evidence of the performance of the system. The presented test case shows that the system has an entity extraction accuracy of 75 percent. This is to state that it recognizes three out of four items anticipated in the requirement description appropriately. The outcome of this experiment demonstrates that the system is capable of capturing most of the significant

schema elements in the textual input, and also demonstrates that the work can be improved in terms of the ability to deal with implicitly expressed entities.

Table 4.2 includes a wider comparison of the three stages of the project.

Table 4.2 Project-Level Overview Comparison

Aspect	IPLA	IPLA 2.0	Schema Design Agent
Problem Domain	AI-powered personal learning assistant for LMS	Multilingual, predictive and adaptive LMS assistant	Automated natural-language to database schema generation
Core Technology	LangChain, FAISS, OpenAI LLM, Moodle	RAG, FAISS, GPT, Random Forest, Moodle APIs	Ollama LLM, FastAPI, Streamlit, Graphviz
Input Type	Text queries with PDFs and URLs	Text and voice input with multilingual support	Natural-language project description
Output Type	Grounded text responses	Multilingual responses with risk prediction	SQL DDL, ER diagram, entity list
Evaluation Method	Functional interface testing	Machine learning metrics	Entity extraction accuracy
Best Result	Fully functional LMS system	Improved predictive performance	Accurate schema generation
Unique Feature	Document-based Q&A	Multilingual support with predictive analytics	Automated schema generation pipeline

Table 4.2 demonstrates how the project has evolved to retrieval-based academic help to predictive educational support and ultimately, the automated database schema generation. This analogy shows how the technical breadth of the work was rising and validates that each next stage added a new level of intelligence and a systematized production. Table 4.3 summarizes the feature-wise development of the system through various stages.

Table 4.3 summarizes the feature-wise development of the system through various stages.

Table 4.3 Feature Enhancement Comparison

Feature	IPLA	IPLA 2.0	Nature of Change
Language Support	English only	Multilingual	Expanded accessibility
Voice Interaction	Not supported	Supported	New interaction mode
AI Response Method	Basic retrieval	Enhanced RAG	Improved contextual accuracy
Predictive Analytics	Not available	Available	Added analytical capability
At-Risk Detection	Not available	Available	Early intervention support
Analytics Dashboard	Not available	Available	Monitoring capability
LMS Integration	Basic	Advanced	Improved connectivity
Evaluation Approach	Functional testing	Quantitative metrics	Improved measurement

Table 4.3 points out the way new functionalities were implemented in subsequent stages of the project. The change between the IPLA and IPLA 2.0 demonstrates that the system was improved by means of increasing the accessibility, analytics, and generating

contextual responses. This uninterrupted development allowed the technical basis of the ultimate schema design agent.

In general, the analysis of results and the results of the test cases indicate that the suggested project works efficiently in all stages. The former levels indicate high retrieval-based academic assistance, multilingual interaction, and predictive analytics and the last stage verifies the fact that the system is able to convert natural-language software requirements into formal database schema artifacts. The figures used in the whole section offer direct visual data on the outputs, and the validation outcome will offer a measurable validation of the system effectiveness. Collectively, these findings prove the fact that the suggested work is technically functional, practically useful and can be applied both in academic demonstration and in the prototype level.

4.3 Observations from Work:

The activity conducted in this project made some valuable observations about the behavior, strengths and weaknesses of intelligent systems at various developmental stages. As the project was developed out of retrieval based academic assistance then to multilingual predictive support and lastly to automated database schema generation, the observations made do not pertain to one module only. Rather, they indicate how various types of artificial intelligence can be combined into a single larger workflow and how their quality of performance is contingent on the quality of input and the design of the system.

Among the most significant things that were observed during the work is that the input quality has a significant impact on the output quality. During the stages of academic assistance, queries posed in writing elicited more precise and context-specific responses, whereas the queries that were not clearly stated or were too broad yielded less contextualized responses. The same trend was seen during the schema generating stage, in which requirement descriptions that directly referenced domain objects and interactions resulted in better entity extraction and more precise relational organization. This means that systems that are based on natural languages are most effective when the input is precise, clear and detailed enough.

The second notable finding is that retrieval-based techniques can significantly enhance the relevance of generated answers. During the learning assistant phases, PDF content, web content, and retrieval-based grounding decreased the unsupported responses and enhanced the reliability of the academic outputs. This implies that language generation should be accompanied by support to retrieval rather than text generation in general. It can also be applied to software design work, as interpretation of requirement in this context can be more reliable when the system is run based on patterns that can be recognized, and not by mere generation.

It is also demonstrated in the work that the multilingual interaction makes the system more practical. Introducing the idea of regional language input to IPLA 2.0 proved that the system can enhance accessibility without distorting the overall meaning of user queries. It was also noted though that variation due to translation can occasionally influence the precise structure of the answer, particularly when the query is complicated or very contextual. This shows that multilingual care enhances access and usability, but an extra layer of interpretation is created and needs to be approached with caution.

The other important observation is associated with predictive analytics. The Logistic Regression and the Random Forest were compared, revealing that, when classification is required, the use of the ensemble-based models is better in the case of educational performance data. Random Forest gave more balanced results in terms of evaluation measures and was particularly successful in enhancing recall and overall classification stability. This implies that model selection is a significant factor in predictive modules, and that more powerful classifiers can help decision-support systems greatly when the data has intricate patterns.

The schema generation stage came up with some of the most important findings in the whole project. One of the obvious conclusions is that the system is efficient in detecting major entities that are explicitly mentioned in the requirement description. The extraction process is generally good when the problem statement explicitly identifies significant components of the system and the resultant schema are uniform across the entities, relationships, SQL output and ER representation. This proves that the system can retrieve the central design structure of a software program out of plain-language input.

Simultaneously, indirectly expressed relationships and implicit entities are also challenging as demonstrated in the work. When the requirement description presupposes some domain knowledge without making it explicit, the system might fail to extract those items. This has an impact on entity completeness and precision of the schema generated. This fact is significant since it demonstrates that the natural-language schema generation is not merely a technical extraction task, but a semantic interpretation one. It points out the necessity of the future advancements of the contextual reasoning and domain knowledge.

The other interesting observation is the uniformity of the workflow in various forms of output. The objects that are derived out of the requirement description are re-occurrence in the relationship output, the SQL DDL statements and the ER diagram. This consistency suggests that the system pipeline is internally consistent, and that the output of one stage allows the next stage to be meaningful. This kind of consistency is needed to ensure that the system is reliable as a design-support tool, since any fragmented or contradictory outputs would decrease the confidence in the produced schema.

The work also shows that the integration of several modules within a layered architecture enhances the usefulness of the system as a whole. The project does not distinguish between entity extraction, relationship detection, SQL generation, and visualization, but instead, these tasks are performed within a single workflow. This connected, yet modular design enables the system to be interpretable, scalable, and more easily enhanced. It was noted that this type of integration renders the system complete and more practical than methods that only address a single aspect of the problem.

The other significant observation is that quantifiable analysis aids in knowing the system in a more realistic way. Entity extraction accuracy used during the schema generation phase enabled the evaluation of the system effectiveness in objective terms as compared to the dependence on visual examination. The validation result of 75 percent indicates that the system is effective, though, it can be improved. This renders evaluation a useful aspect of the project since it facilitates the performance analysis as well as successive refinement.

In terms of implementation, the project also demonstrates that the modest software and hardware resources have the capability to create a useful intelligent design assistant. The proposed work can work in a normal academic development environment as opposed to computation-intensive system of image-processing. This will make it easier to use by students, display in classrooms, and at a prototype level.

In general, the experiences of the work support the idea that the proposed project is effective in proving the practical usefulness of AI-assisted systems in educational support and automation of software design. Simultaneously, the work emphasizes the value of proper input, organization of the workflow, grounding of retrieval, assistance in evaluation, and further enhancement of semantic comprehension. These findings give a solid foundation to further development and make the project a valuable prototype and a valuable contribution to the academic field.

CHAPTER-5

Conclusion and Future work

5.1 Conclusion :-

The AI-based Automated Database Schema Design Agent introduces a formatted and realistic method of converting natural-language software requirement statements to formal database design artifacts. The project was created in several steps starting with retrieval-based academic support, moving on to multilingual and predictive learning support, and finally becoming a system with the ability to generate the schema on its own. The development demonstrates that the work is not a single implementation, but the consequence of the continuous development of the related intelligent system implementation.

The last stage of the project shows how it is possible to read software requirement descriptions written in natural language and interpret them in a meaningful manner to deduce entities, relationships, SQL schema structures, and ER-style visual outputs. The system is able to accept requirement text, identify domain-relevant elements, identify structural relationships, and create implementation-oriented artifacts. This affirms that the suggested workflow can be used to bridge the gap between informal descriptions of requirements and formal relational database design.

The outcome of schema generation module indicates that the system is able to extract the key actors in a problem statement and arrange them into a logical relational model. The resulting SQL Data Definition Language statements, and ER-like diagrams, additionally demonstrate that the result is not only conceptual but also practical. The presented example shows that the approach is effective to capture most of the expected schema elements based on the textual input, which is proven by the validation process, which achieved an entity extraction accuracy of 75 percent.

A second value that the project has brought about is the consolidation of a combination of steps into one end-to-end workflow. Unlike treating requirement interpretation, entity extraction, relationship detection, SQL generation, visualization and evaluation as independent processes, the proposed system unites them into a single system. This

increases the consistency of output and utility of the system as a practical design-support tool.

The project also shows how the use of artificial intelligence together with the existing principles of database design is valuable. Although the system relies on the use of language-based analysis in interpretation of requirement descriptions, it does not avoid using concepts of relational design like entities, attributes, keys, and relationship. This tradeoff between AI-assisted processing and designed design methodology enhances the reliability, as well as interpretability of the system.

In general, the project demonstrates that AI-assisted methods can have a significant role in software engineering projects, including database schema design. The system also gives a powerful first draft, even though the generated schema will still need human inspection and improvement in some situations, which takes less time than handwriting, increases explicitness of design and helps with academic study, as well as with prototype-level development. The work thus is a valuable addition in the field of smart software design automation.

5.2 Future Study :-

Though the proposed system yields valuable output, various areas can be investigated in further research to make the system more accurate, flexible, and practical to use. The enhancement of semantic understanding, especially the work with implicit entities and indirectly conveyed relationships is one of the most crucial directions of future work. Currently, the system works well with key domain elements well defined in the requirement description, but future releases can be developed to be able to better represent hidden or assumed notions.

The other potential direction to take on in future research is adding normalization support. The existing system is primarily concerned with entity extraction, relation mapping as well as schema generation, although it can be enhanced in the future by looking at the generated schema in terms of normalization quality and redundancy. The usefulness of the generated schema would be enhanced by the addition of checks on first normal form, second normal form, and third normal form, and making it useful in developing a practical database.

It is also possible to extend the system with new sophisticated validation techniques. Currently the measure of performance is primarily made through the accuracy of entity extraction. The assessment of relationship correctness, key assignment quality, attribute completeness and overall structural validity may be included in future work. A more comprehensive assessment model would give a more comprehensive picture of system performance and allow distinguishing weak points more accurately.

Interactive refinement is another significant direction. Under the existing workflow, the system will automatically generate the outputs and present the same to the user to review. It might be possible to enable users to make edits, approve, reject or modify extracted entities and relationships in the interface itself in the future. Human-in-the-loop refinement would also enhance usability and increase the ability of the system to adapt to the real-world software design requirements.

There is also room to further investigate the support of other SQL dialects including MySQL, PostgreSQL and SQLite. Currently, the SQL generated is a general relational form but this can be enhanced by offering the user options on what database platform to use and then presenting SQL statements specific to that platform.

Multilingual requirement processing of schema generation itself is another helpful direction. Although multilingual support has already been investigated during the learning assistant stage of the project, the same can be applied to the schema design module in order to be able to interpret requirement descriptions in different languages as well to generate a database. This would greatly enhance accessibility and increase the practical applicability of the system.

Future research can also study the use of knowledge-assisted or retrieval-grounded schema generation. The system can possibly enhance its ambiguity management and produce more dependable outputs on complex software domains by taking into account domain knowledge bases or stored design examples. This kind of integration would enhance contextual reasoning and minimize the chances of overlooking crucial elements of schema.

Lastly, the project can be furthered to a more comprehensive intelligent software design assistant by linking schema generation to subsequent steps like query template

generation, API structure suggestion or software module design support. This would enable the system to stop working on database design and integrate into a greater workflow of AI-assisted development.

Generally, further research can be oriented to better contextualization, structural accuracy, further development of assessment techniques, interactive refinement, and generalized practical assistance at various fields and database systems. These guidelines would enhance the suggested system and assist in changing it to a higher level of an advanced and dependable intelligent software engineering tool.

References:

- [1] Xu, T., Li, B.: Knowledge-Enhanced Label-Wise Large Language Model for Safe and Interpretable Drug Recommendation. *Electronics (Switzerland)*, 14(1), (2025)
- [2] Tao, S., Yang, Y., Liu, X., Feng, Y., Zheng, J.: Learning Universal Knowledge Graph Embedding for Predicting Biomedical Pairwise Interactions, (2025)
- [3] Ling Ong, J. C., Michael, C., Ng, N., Elangovan, K., Ting Tan, N. Y., Jin, L., Xie, Q., Wei Ting, D. S., Rodriguez-Monguio, R., Bates, D. W., Liu, N.: Generative AI and Large Language Models in Reducing Medication Related Harm and Adverse Drug Events – A Scoping Review, (2024)
- [4] Xu, C., Bulusu, K. C., Pan, H., Elemento, O.: DDI-GPT: Explainable Prediction of Drug-Drug Interactions using Large Language Models enhanced with Knowledge Graphs, (2024)
- [5] Im, S., Ko, Y.: LLM-Enhanced Multimodal Framework for Drug–Drug Interaction Prediction. *Biomedicines*, 13(10), 2355, (2025)
- [6] Latif-Martínez, H., Suárez-Varela, J., Cabellos-Aparicio, A., Barlet-Ros, P.: GAT-AD: Graph Attention Networks for Contextual Anomaly Detection in Network Monitoring. *Computers & Industrial Engineering*, 200, 110830, (2025)
- [7] Ryu, J. Y., Kim, H. U., Lee, S. Y.: Deep Learning Improves Prediction of Drug–Drug and Drug–Food Interactions. *Proceedings of the National Academy of Sciences*, 115(18), E4304–E4311, (2018)
- [8] Kha, Q.-H., Le, V.-H., Hung, T. N. K., Nguyen, N. T. K., Le, N. Q. K.: Development and Validation of an Explainable Machine Learning-Based Prediction Model for Drug–Food Interactions from Chemical Structures. *Sensors*, 23(8), 3962, (2023)
- [9] Yang, B., Song, D., Li, Y., et al.: Drug-Drug Interaction Prediction of Traditional Chinese Medicine Based on Graph Attention Networks. *Scientific Reports*, 15, 18635, (2025)

- [10] Kotkondawar, R. R., Sutar, S. R., Kiwelekar, A. W., et al.: A Generative Framework for Enhancing Drug Target Interaction Prediction in Drug Discovery, (Year not specified)
- [11] Rajkumar, R., Li, J., Liang, P.: Evaluating Large Language Models Trained on Code. *arXiv*, (2022)
- [12] Zhong, R., Yu, T., Klein, D.: Semantic Evaluation for Text-to-SQL with Execution Accuracy. *arXiv*, (2020)
- [13] Brown, T. B., et al.: Language Models are Few-Shot Learners. *NeurIPS*, (2020)
- [14] Devlin, J., Chang, M. W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*, (2019)
- [15] Yu, T., et al.: Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP*, (2018)
- [16] Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, (2020)
- [17] Gao, L., Dai, Z., Pasupat, P., et al.: RARR: Researching and Revising What Language Models Say, (2023)
- [18] Yue, T., Briand, L. C., Labiche, Y.: A Systematic Review of Transformation Approaches Between User Requirements and Analysis Models. *Requirements Engineering*, (2011)
- [19] Deng, X., Zhang, X., Ma, H., et al.: Hybrid NLP and Large Language Model Frameworks for Structured Software Artifact Generation, (2024)
- [20] Shneiderman, B.: Human-Centered AI. *Oxford University Press*, (2022)

APPENDIX

```
from fastapi import FastAPI
```

```
from pydantic import BaseModel
```

```
from backend.utils import (
```

```
    extract_entities,
```

```
    generate_sql,
```

```
    handle_many_to_many
```

```
)
```

```
from backend.er import generate_er
```

```
app = FastAPI()
```

```
class Request(BaseModel):
```

```
    text: str
```

```
@app.post("/generate")
```

```
def generate_schema(req: Request):
```

```
    try:
```

```
        entities, relationships, explanation, suggestions = extract_entities(req.text)
```

```

if not entities:

    return {"error": "Failed to generate schema"}

entities = handle_many_to_many(entities, relationships)

sql = generate_sql(entities)

try:

    er_path = generate_er(entities, relationships)
except:

    er_path = None

return {

    "schema": entities,

    "relationships": relationships,

    "sql": sql,

    "er_diagram": er_path,

    "explanation": explanation,

    "suggestions": suggestions

}

except Exception as e:

```



```
return {  
    "error": "Internal server error",  
    "details": str(e)  
}
```

17% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

 30 AI-generated only 17%
Likely AI-generated text from a large-language model.

 0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Crossref database
- Crossref posted content database

Match Groups

-  **80 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **7 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.